

Data Analyst SaaS — Implementation Manual (Python backend + Next.js frontend)

Product: Data Analyst SaaS (iTarang)
Scope: Part A — Python agent service (`analyst-agent` , FastAPI + LangGraph), sections 2–9. Part B — Next.js + TypeScript product shell (`analyst-web`), sections 10–15.
Version: 1.0 — 10 September 2026
Audience: a backend developer starting with zero context on this project

Table of Contents

Data Analyst SaaS — Implementation Manual (Python backend + Next.js frontend)

- 1. What you are building
 - 1.1 Architecture at a glance
 - 1.2 Build order (from the development plan)
- PART A — Python backend (analyst-agent)
- 2. Environment setup
 - 2.1 Prerequisites
 - 2.2 Virtual environment choice
 - 2.3 Dependency management with pyproject.toml
 - 2.4 Install and lock
 - 2.5 Local infrastructure with Docker Compose
- 3. Project structure
- 4. Framework: FastAPI installation and configuration
 - 4.1 Settings
 - 4.2 Structured logging
 - 4.3 LLM client
 - 4.4 Application factory
- 5. Database configuration
 - 5.1 App DB — SQLAlchemy engine and session
 - 5.2 App DB — models
 - 5.3 Migrations with Alembic
 - 5.4 Checkpoint DB — LangGraph
 - 5.5 Customer DB — read-only role and demo data
 - 5.6 Customer DB — connector and schema introspection
- 6. API development
 - 6.1 Authentication and tenant context
 - 6.2 Credential vault
 - 6.3 Request / response schemas
 - 6.4 Connections service and routes
 - 6.5 Runs route (SSE)
- 7. Business logic — the LangGraph agent
 - 7.1 State
 - 7.2 Prompts
 - 7.3 Router node
 - 7.4 SQL generation node
 - 7.5 SQL guard node — code, not LLM
 - 7.6 Execution and answer nodes
 - 7.7 Web tool node (Playwright)
 - 7.8 Building the graph
 - 7.9 Run service — orchestration and usage
 - 7.10 End-to-end checkpoint
 - 7.11 Golden evals
- 8. Testing
 - 8.1 Configuration
 - 8.2 Unit tests
 - 8.3 Integration tests
- 9. Deployment preparation
 - 9.1 Environment variables in production
 - 9.2 Dockerfile
 - 9.3 Production Compose
 - 9.4 Logging in production
 - 9.5 CI/CD (GitHub Actions)
 - 9.6 Production readiness checklist
- PART B — Next.js + TypeScript frontend (analyst-web)
- 10. Frontend overview — Next.js + TypeScript
 - 10.1 Responsibilities split
 - 10.2 Stack
- 11. Frontend environment setup
 - 11.1 Prerequisites

- 11.2 Create the project
 - 11.3 Environment variables
 - 12. Frontend project structure
 - 13. Auth, tenants and the agent token
 - 13.1 Supabase tables
 - 13.2 Supabase clients
 - 13.3 Resolve the tenant
 - 13.4 Mint the agent JWT (server only)
 - 14. Agent client, connections and chat
 - 14.1 Generate the typed client
 - 14.2 Connections — server-side proxy route
 - 14.3 SSE parser for POST streams
 - 14.4 Run state machine hook
 - 14.5 Chat UI
 - 15. Frontend testing, billing and deployment
 - 15.1 Vitest
 - 15.2 Playwright e2e
 - 15.3 Billing hooks (phase 6)
 - 15.4 Dockerfile and deploy
 - 15.5 Frontend readiness checklist
 - 16. Appendix
 - 16.1 Command cheat sheet
 - 16.2 Extension points (phase 5)
 - 16.3 Troubleshooting index
-

1. What you are building

A multi-tenant service that answers business questions over a customer's own data. A customer connects a data source (a Postgres database, a web dashboard such as Intellicar, or an uploaded file). They ask a question in plain English. The service:

1. routes the question to the right tool,
2. writes a read-only SQL query (or scrapes a dashboard),
3. validates and executes it against **that tenant's** source only,
4. streams back an answer with the numbers, the SQL used, and a table.

The Next.js product shell (auth, billing, chat UI) is a separate repository and is out of scope here. It calls this service over one HTTP endpoint and reads a Server-Sent-Events (SSE) stream.

1.1 Architecture at a glance



1.2 Build order (from the development plan)

Phase	Deliverable	"Done" means
0	Repo, Docker Compose, stub graph, SSE endpoint	<code>curl /runs</code> streams a stub answer, visible in LangSmith
1	SQL tool, single tenant, guard, golden evals	25/30 golden questions pass on your own DB
2	Auth, vault, connections API	second person connects a DB without you
3	Web tool (Playwright)	Intellicar live query works for two tenants
4	Queue/workers, limits, usage, Docker deploy	killing a worker mid-run gives a clean error
5	File tool (DuckDB), charts	spreadsheet-only customer gets value

Part A covers phases 0–2 fully, phase 3 in section 7.7 and phase 4 in section 9; Part B (sections 10–15) covers the Next.js product shell for phases 2, 4 and 6. Phase 5 is a documented extension point (section 16.2).

2. Environment setup

2.1 Prerequisites

Tool	Version	Check
Python	3.12.x	<code>python3 --version</code>
Docker + Compose	24+ / v2	<code>docker compose version</code>
Git	any	<code>git --version</code>
PostgreSQL client	15+	<code>psql --version</code>

Install Python 3.12

Ubuntu / Debian

```
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository -y ppa:deadsnakes/ppa
sudo apt install -y python3.12 python3.12-venv python3.12-dev build-essential libpq-dev
python3.12 --version
```

Expected output: `Python 3.12.x`

macOS (Homebrew)

```
brew install python@3.12 postgresql@16
python3.12 --version
```

Windows — install from python.org, tick "Add Python to PATH", then use `py -3.12` instead of `python3.12` in every command below. Use WSL2 (Ubuntu) if possible; Playwright and Docker behave better there.

2.2 Virtual environment choice

We use the standard library `venv` plus a `pyproject.toml`. Reasons: zero extra tooling on the production VPS, identical behaviour in Docker, and `pip install -e .` gives you an importable `app` package. Poetry and conda are acceptable on a laptop but are **not** used in CI or Docker.

```
mkdir -p ~/code/analyst-agent && cd ~/code/analyst-agent
git init
python3.12 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
```

Checkpoint: which python prints a path ending in `analyst-agent/.venv/bin/python`.

2.3 Dependency management with `pyproject.toml`

Create the file below. Pin major versions; let pip resolve minors. A lock file is generated in section 2.4.

File: `pyproject.toml`

```
[build-system]
requires = ["setuptools>=69", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "analyst-agent"
version = "0.1.0"
description = "Multi-tenant data analyst agent service (FastAPI + LangGraph)"
requires-python = ">=3.12"
dependencies = [
    "fastapi>=0.115",
    "uvicorn[standard]>=0.30",
    "pydantic>=2.8",
    "pydantic-settings>=2.4",
    "sse-starlette>=2.1",
    "python-jose[cryptography]>=3.3",
    "cryptography>=43",
    "sqlalchemy>=2.0",
    "psycopg[binary,pool]>=3.2",
    "alembic>=1.13",
    "sqlglot>=25",
    "langchain>=0.3",
    "langchain-core>=0.3",
    "langchain-openai>=0.2",
    "langgraph>=0.2",
    "langgraph-checkpoint-postgres>=2.0",
    "langsmith>=0.1",
    "structlog>=24",
    "playwright>=1.47",
    "duckdb>=1.1",
    "pandas>=2.2",
    "redis>=5",
]

[project.optional-dependencies]
dev = [
    "pytest>=8",
    "pytest-asyncio>=0.24",
    "pytest-cov>=5",
    "httpx>=0.27",
    "ruff>=0.6",
    "mypy>=1.11",
    "testcontainers[postgres]>=4.8",
    "pip-tools>=7",
]

[tool.setuptools.packages.find]
include = ["app*"]

[tool.ruff]
line-length = 100
target-version = "py312"

[tool.pytest.ini_options]
asyncio_mode = "auto"
testpaths = ["tests"]
addopts = "-q --cov=app --cov-report=term-missing"
```

2.4 Install and lock

```
pip install -e ".[dev]"
pip-compile --extra dev -o requirements.lock pyproject.toml # reproducible lock
playwright install chromium # browser for the web tool
playwright install-deps chromium # Linux system libs (needs sudo on bare hosts)
```

Checkpoint:

```
python -c "import fastapi, langgraph, sqlglot; print('ok')"
```

Expected: ok.

Common errors

Symptom	Fix
psycopg build error mentioning pg_config	sudo apt install libpq-dev (Linux) / brew install postgresql (macOS)
playwright install-deps asks for sudo in CI	Use the Playwright Docker base image (see section 9.2) or run it as root in the Dockerfile
pip-compile: command not found	You skipped .[dev]; run pip install pip-tools

2.5 Local infrastructure with Docker Compose

Two Postgres databases (app + checkpoint), one Redis. Keep them separate from day one so the checkpoint DB can be wiped without touching tenants.

File: docker-compose.dev.yml

```
services:
  app-db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: analyst
    ports: ["5433:5432"]
    volumes: ["app-db:/var/lib/postgresql/data"]

  checkpoint-db:
    image: postgres:16
    environment:
      POSTGRES_USER: ckpt
      POSTGRES_PASSWORD: ckpt
      POSTGRES_DB: checkpoints
    ports: ["5434:5432"]
    volumes: ["ckpt-db:/var/lib/postgresql/data"]

  redis:
    image: redis:7
    ports: ["6379:6379"]

# A throw-away "customer" database to develop against
demo-customer-db:
  image: postgres:16
  environment:
    POSTGRES_USER: demo
    POSTGRES_PASSWORD: demo
    POSTGRES_DB: demo
  ports: ["5435:5432"]
  volumes: [".:/scripts/demo_customer.sql:/docker-entrypoint-initdb.d/init.sql:ro"]

volumes:
  app-db: {}
  ckpt-db: {}
```

```
docker compose -f docker-compose.dev.yml up -d
docker compose -f docker-compose.dev.yml ps
```

Expected: four containers in `running` state.

Checkpoint:

```
psql postgresql://app:app@localhost:5433/analyst -c "select 1"
```

Expected: a single row with `1`.

3. Project structure

```
analyst-agent/
├── pyproject.toml
├── requirements.lock
├── .env.example
├── .env
├── docker-compose.dev.yml
├── docker-compose.yml
├── Dockerfile
├── alembic.ini
├── alembic/
│   ├── env.py
│   └── versions/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── config.py
│   ├── logging.py
│   ├── llm.py
│   └── api/
│       ├── __init__.py
│       ├── deps.py
│       ├── schemas.py
│       ├── routes_health.py
│       ├── routes_connections.py
│       └── routes_runs.py
│   ├── db/
│       ├── __init__.py
│       ├── session.py
│       └── models.py
│   ├── security/
│       ├── __init__.py
│       ├── auth.py
│       └── vault.py
│   ├── connectors/
│       ├── __init__.py
│       ├── base.py
│       ├── postgres.py
│       └── registry.py
│   ├── agent/
│       ├── __init__.py
│       ├── state.py
│       ├── graph.py
│       ├── prompts.py
│       └── nodes/
│           ├── __init__.py
│           ├── router.py
│           ├── sql_gen.py
│           ├── sql_guard.py
│           ├── db_exec.py
│           ├── web_tool.py
│           └── answer.py
│   ├── services/
│       ├── __init__.py
│       ├── connections.py
│       ├── runs.py
│       └── errors.py
│   └── workers/
│       └── web_session.py
├── evals/
│   ├── golden_sql.yaml
│   └── run_evals.py
├── scripts/
│   ├── demo_customer.sql
│   └── create_readonly_role.sql
├── tests/
│   ├── conftest.py
│   ├── unit/
│   │   ├── test_sql_guard.py
│   │   ├── test_vault.py
│   │   └── test_router.py
│   └── integration/
│       ├── test_connections_api.py
│       └── test_runs_api.py
```

Why this shape

- `api/` knows HTTP only. `services/` knows business rules. `agent/` knows LangGraph. `connectors/` knows customer data sources. Nothing imports upward.
- `security/vault.py` is the only module allowed to see plaintext credentials.
- Every function under `connectors/` and `agent/` takes a `tenant_id` argument. There is no default tenant anywhere.

Create the skeleton:

```
mkdir -p app/{api,db,security,connectors,agent/nodes,services,workers} alembic/versions evals scripts tests/{unit,integration}
touch app/__init__.py app/api/__init__.py app/db/__init__.py app/security/__init__.py \
    app/connectors/__init__.py app/agent/__init__.py app/agent/nodes/__init__.py \
    app/services/__init__.py app/workers/__init__.py tests/__init__.py
```

4. Framework: FastAPI installation and configuration

FastAPI is already installed via `pyproject.toml`. This section wires configuration, logging, and the application factory.

4.1 Settings

File: `app/config.py`

```
from functools import lru_cache
from pydantic import Field, PostgresDsn, SecretStr
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")

    env: str = Field(default="dev", alias="APP_ENV")
    log_level: str = "INFO"

    # LLM
    openrouter_api_key: SecretStr
    openrouter_model: str = "openai/gpt-4o-mini"
    openrouter_base_url: str = "https://openrouter.ai/api/v1"

    # Databases
    app_db_url: PostgresDsn
    checkpoint_db_url: PostgresDsn

    # Security
    credential_encryption_key: SecretStr          # Fernet key, 32 url-safe base64 bytes
    jwt_secret: SecretStr                        # shared with Next.js
    jwt_algorithm: str = "HS256"

    # Limits
    max_rows: int = 500
    statement_timeout_ms: int = 8000
    max_sql_retries: int = 2

    # Web tool
    session_store_dir: str = "./sessions"
    playwright_headless: bool = True

    # Tracing
    langsmith_tracing: bool = False
    langsmith_api_key: SecretStr | None = None
    langsmith_project: str = "analyst-agent-dev"

    @lru_cache
    def get_settings() -> Settings:
        return Settings()
```

File: `.env.example` (commit this; never commit `.env`)

```
APP_ENV=dev
LOG_LEVEL=INFO

OPENROUTER_API_KEY=sk-or-...
OPENROUTER_MODEL=openai/gpt-4o-mini

APP_DB_URL=postgresql+psycpg://app:app@localhost:5433/analyst
CHECKPOINT_DB_URL=postgresql://ckpt:ckpt@localhost:5434/checkpoints

CREDENTIAL_ENCRYPTION_KEY= # python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
JWT_SECRET=change-me-32-chars-minimum

MAX_ROWS=500
STATEMENT_TIMEOUT_MS=8000

SESSION_STORE_DIR=./sessions
PLAYWRIGHT_HEADLESS=true

LANGSMITH_TRACING=true
LANGSMITH_API_KEY=lsv2...
LANGSMITH_PROJECT=analyst-agent-dev
```

```
cp .env.example .env
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
# paste the output into CREDENTIAL_ENCRYPTION_KEY in .env
echo ".env
.venv/
sessions/
__pycache__/
*.pyc
.pytest_cache/
.coverage" > .gitignore
```

Checkpoint:

```
python -c "from app.config import get_settings; s=get_settings(); print(s.env, s.openrouter_model)"
```

Expected: `dev openai/gpt-4o-mini`. A `ValidationError` naming a field means that env var is missing.

4.2 Structured logging

File: `app/logging.py`

```
import logging
import structlog
from app.config import get_settings

def configure_logging() -> None:
    settings = get_settings()
    level = getattr(logging, settings.log_level.upper(), logging.INFO)
    logging.basicConfig(level=level, format="%(message)s")

    processors = [
        structlog.contextvars.merge_contextvars,
        structlog.processors.add_log_level,
        structlog.processors.TimeStamper(fmt="iso"),
    ]
    if settings.env == "dev":
        processors.append(structlog.dev.ConsoleRenderer())
    else:
        processors.append(structlog.processors.JSONRenderer())

    structlog.configure(
        processors=processors,
        wrapper_class=structlog.make_filtering_bound_logger(level),
        cache_logger_on_first_use=True,
    )

log = structlog.get_logger()
```

Every request binds `tenant_id` and `run_id` into `structlog.contextvars`, so every log line downstream carries them without passing arguments around.

4.3 LLM client

File: `app/llm.py`

```
import os
from langchain_openai import ChatOpenAI
from app.config import get_settings

def configure_tracing() -> None:
    s = get_settings()
    if s.langsmith_tracing and s.langsmith_api_key:
        os.environ["LANGSMITH_TRACING"] = "true"
        os.environ["LANGSMITH_API_KEY"] = s.langsmith_api_key.get_secret_value()
        os.environ["LANGSMITH_PROJECT"] = s.langsmith_project

def get_llm(temperature: float = 0.0) -> ChatOpenAI:
    s = get_settings()
    return ChatOpenAI(
        model=s.openrouter_model,
        api_key=s.openrouter_api_key.get_secret_value(),
        base_url=s.openrouter_base_url,
        temperature=temperature,
        timeout=60,
        max_retries=2,
    )
```

4.4 Application factory

File: `app/main.py`

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.config import get_settings
from app.logging import configure_logging, log
from app.llm import configure_tracing
from app.api import routes_health, routes_connections, routes_runs

@asynccontextmanager
async def lifespan(app: FastAPI):
    configure_logging()
    configure_tracing()
    log.info("startup", env=get_settings().env)
    yield
    log.info("shutdown")

def create_app() -> FastAPI:
    app = FastAPI(title="Analyst Agent", version="0.1.0", lifespan=lifespan)
    app.add_middleware(
        CORSMiddleware,
        allow_origins=["http://localhost:3000"], # Next.js dev; tighten in prod
        allow_methods=["*"],
        allow_headers=["*"],
    )
    app.include_router(routes_health.router)
    app.include_router(routes_connections.router, prefix="/connections", tags=["connections"])
    app.include_router(routes_runs.router, prefix="/runs", tags=["runs"])
    return app

app = create_app()
```

File: `app/api/routes_health.py`

```
from fastapi import APIRouter

router = APIRouter()

@router.get("/health", tags=["ops"])
def health() -> dict:
    return {"status": "ok"}
```

Create empty placeholders for the other two routers so the app imports:

```
printf 'from fastapi import APIRouter\nrouter = APIRouter()\n' > app/api/routes_connections.py
printf 'from fastapi import APIRouter\nrouter = APIRouter()\n' > app/api/routes_runs.py
```

Run it:

```
uvicorn app.main:app --reload --port 8000
```

Checkpoint:

```
curl -s localhost:8000/health
```

Expected: `{"status": "ok"}`. Open `http://localhost:8000/docs` — Swagger UI lists `/health`.

Common errors

Symptom	Fix
ModuleNotFoundError: app	You are not in the project root, or <code>pip install -e .</code> was skipped
Address already in use	<code>lsof -i :8000</code> and kill the process, or use <code>--port 8001</code>
Settings ValidationError at startup	<code>.env</code> missing a required key; compare with <code>.env.example</code>

5. Database configuration

There are **three** kinds of database in this service. Do not confuse them.

Database	Owner	Purpose	Access
App DB	us	tenants, connections, runs, usage	read/write, migrated by Alembic
Checkpoint DB	us	LangGraph thread state (chat memory)	managed by LangGraph, no Alembic
Customer DB	the tenant	the data being analysed	read-only , never migrated, never written

5.1 App DB — SQLAlchemy engine and session

File: `app/db/session.py`

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, Session
from app.config import get_settings

_engine = create_engine(
    str(get_settings().app_db_url),
    pool_pre_ping=True,
    pool_size=5,
    max_overflow=10,
)
SessionLocal = sessionmaker(bind=_engine, autoflush=False, expire_on_commit=False)

def get_db():
    """FastAPI dependency: one session per request."""
    db: Session = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

5.2 App DB — models

File: `app/db/models.py`

```

import uuid
from datetime import datetime
from sqlalchemy import String, Text, ForeignKey, Integer, DateTime, JSON, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

def _uuid() -> str:
    return str(uuid.uuid4())

class Tenant(Base):
    __tablename__ = "tenants"
    id: Mapped[str] = mapped_column(String(36), primary_key=True, default=_uuid)
    name: Mapped[str] = mapped_column(String(200))
    plan: Mapped[str] = mapped_column(String(50), default="free")
    daily_token_budget: Mapped[int] = mapped_column(Integer, default=200_000)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())

    connections: Mapped[list["Connection"]] = relationship(back_populates="tenant")

class Connection(Base):
    """A customer data source. `secret_enc` holds the Fernet-encrypted DSN or login JSON."""
    __tablename__ = "connections"
    id: Mapped[str] = mapped_column(String(36), primary_key=True, default=_uuid)
    tenant_id: Mapped[str] = mapped_column(ForeignKey("tenants.id"), index=True)
    name: Mapped[str] = mapped_column(String(200))
    kind: Mapped[str] = mapped_column(String(20)) # 'postgres' | 'web' | 'file'
    secret_enc: Mapped[str] = mapped_column(Text)
    schema_cache: Mapped[dict | None] = mapped_column(JSON, nullable=True)
    schema_cached_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True), nullable=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())

    tenant: Mapped[Tenant] = relationship(back_populates="connections")

class Run(Base):
    __tablename__ = "runs"
    id: Mapped[str] = mapped_column(String(36), primary_key=True, default=_uuid)
    tenant_id: Mapped[str] = mapped_column(ForeignKey("tenants.id"), index=True)
    connection_id: Mapped[str] = mapped_column(ForeignKey("connections.id"))
    thread_id: Mapped[str] = mapped_column(String(100), index=True)
    question: Mapped[str] = mapped_column(Text)
    status: Mapped[str] = mapped_column(String(20), default="running") # running|done|error
    tool: Mapped[str | None] = mapped_column(String(20), nullable=True)
    sql: Mapped[str | None] = mapped_column(Text, nullable=True)
    error: Mapped[str | None] = mapped_column(Text, nullable=True)
    prompt_tokens: Mapped[int] = mapped_column(Integer, default=0)
    completion_tokens: Mapped[int] = mapped_column(Integer, default=0)
    rows_returned: Mapped[int] = mapped_column(Integer, default=0)
    duration_ms: Mapped[int] = mapped_column(Integer, default=0)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())

```

Run doubles as the usage ledger — tokens, rows, duration per run per tenant. This is the data you will price from.

5.3 Migrations with Alembic

```
alembic init alembic
```

Edit `alembic.ini`: leave `sqlalchemy.url` blank; it is read from settings.

Replace `alembic/env.py` with:

```
from logging.config import fileConfig
from alembic import context
from sqlalchemy import engine_from_config, pool

from app.config import get_settings
from app.db.models import Base

config = context.config
if config.config_file_name:
    fileConfig(config.config_file_name)

config.set_main_option("sqlalchemy.url", str(get_settings().app_db_url))
target_metadata = Base.metadata

def run_migrations_offline() -> None:
    context.configure(url=config.get_main_option("sqlalchemy.url"),
                      target_metadata=target_metadata, literal_binds=True)
    with context.begin_transaction():
        context.run_migrations()

def run_migrations_online() -> None:
    connectable = engine_from_config(config.get_section(config.config_ini_section, {}),
                                    prefix="sqlalchemy.", poolclass=pool.NullPool)
    with connectable.connect() as connection:
        context.configure(connection=connection, target_metadata=target_metadata)
        with context.begin_transaction():
            context.run_migrations()

if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()
```

Generate and apply the first migration:

```
alembic revision --autogenerate -m "init tenants connections runs"
alembic upgrade head
```

Expected output ends with Running upgrade -> <hash>, init tenants connections runs.

Checkpoint:

```
psql postgresql://app:app@localhost:5433/analyst -c "\dt"
```

Expected: tables alembic_version, tenants, connections, runs.

Common errors

Symptom	Fix
Autogenerate produces an empty migration	alembic/env.py is not importing app.db.models ; the models must be imported so Base.metadata is populated
Target database is not up to date	Run alembic upgrade head before creating a new revision
psycopg.OperationalError: connection refused	docker compose -f docker-compose.dev.yml up -d and check the port (5433)

5.4 Checkpoint DB — LangGraph

LangGraph creates its own tables. You only need to call setup() once.

```
python - <<'EOF'
from langgraph.checkpoint.postgres import PostgresSaver
from app.config import get_settings
with PostgresSaver.from_conn_string(str(get_settings().checkpoint_db_url)) as saver:
    saver.setup()
print("checkpoint tables ready")
EOF
```

Checkpoint: psql postgresql://ckpt:ckpt@localhost:5434/checkpoints -c "\dt" shows checkpoints, checkpoint_blobs, checkpoint_writes, checkpoint_migrations.

5.5 Customer DB — read-only role and demo data

Never connect to a customer database as an owner. Ship this script to customers (and run it on your own IoT database):

File: scripts/create_readonly_role.sql

```
-- Run as a superuser on the CUSTOMER database. Replace the password.
CREATE ROLE analyst_ro LOGIN PASSWORD 'CHANGE_ME';
GRANT CONNECT ON DATABASE demo TO analyst_ro;
GRANT USAGE ON SCHEMA public TO analyst_ro;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyst_ro;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO analyst_ro;
ALTER ROLE analyst_ro SET statement_timeout = '8s';
ALTER ROLE analyst_ro SET default_transaction_read_only = on;
```

File: `scripts/demo_customer.sql` (seeds the `demo-customer-db` container)

```
CREATE TABLE dealers (
  id SERIAL PRIMARY KEY, name TEXT NOT NULL, city TEXT NOT NULL, created_at DATE NOT NULL);
CREATE TABLE batteries (
  id SERIAL PRIMARY KEY, dealer_id INT REFERENCES dealers(id), imei TEXT UNIQUE NOT NULL,
  chemistry TEXT NOT NULL, capacity_kwh NUMERIC(5,2) NOT NULL, sold_on DATE NOT NULL,
  price_inr NUMERIC(12,2) NOT NULL);
CREATE TABLE telemetry (
  id BIGSERIAL PRIMARY KEY, battery_id INT REFERENCES batteries(id),
  ts TIMESTAMPTZ NOT NULL, soc NUMERIC(5,2), voltage NUMERIC(6,2), temp_c NUMERIC(5,2));

INSERT INTO dealers (name, city, created_at) VALUES
  ('Sharma Motors', 'Gurugram', '2026-01-10'), ('Roy Auto', 'Howrah', '2026-02-14'),
  ('Patil EV', 'Nashik', '2026-03-02');
INSERT INTO batteries (dealer_id, imei, chemistry, capacity_kwh, sold_on, price_inr) VALUES
  (1, '8600000000000001', 'LFP', 7.2, '2026-07-05', 48500), (1, '8600000000000002', 'LFP', 7.2, '2026-08-12', 48500),
  (2, '8600000000000003', 'LFP', 10.0, '2026-08-20', 61000), (3, '8600000000000004', 'LFP', 7.2, '2026-08-28', 47800),
  (3, '8600000000000005', 'LFP', 10.0, '2026-09-01', 60500);
INSERT INTO telemetry (battery_id, ts, soc, voltage, temp_c)
SELECT b.id, now() - (g || ' hours')::interval, 20 + random()*80, 48 + random()*6, 25 + random()*15
FROM batteries b, generate_series(1, 48) g;

CREATE ROLE analyst_ro LOGIN PASSWORD 'ro';
GRANT CONNECT ON DATABASE demo TO analyst_ro;
GRANT USAGE ON SCHEMA public TO analyst_ro;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyst_ro;
ALTER ROLE analyst_ro SET default_transaction_read_only = on;
```

Recreate the demo container so the seed runs:

```
docker compose -f docker-compose.dev.yml rm -sf demo-customer-db
docker compose -f docker-compose.dev.yml up -d demo-customer-db
psql postgresql://analyst_ro:ro@localhost:5435/demo -c "select count(*) from telemetry"
```

Expected: 240.

Verify the role is truly read-only:

```
psql postgresql://analyst_ro:ro@localhost:5435/demo -c "delete from dealers"
```

Expected: `ERROR: cannot execute DELETE in a read-only transaction`. If this succeeds, stop — the role is misconfigured.

5.6 Customer DB — connector and schema introspection

File: `app/connectors/base.py`

```
from typing import Protocol, Any

class Connector(Protocol):
    kind: str
    def describe_schema(self) -> dict[str, Any]: ...
    def run_select(self, sql: str, max_rows: int) -> tuple[list[str], list[tuple]]: ...
```

File: `app/connectors/postgres.py`

```

from typing import Any
from sqlalchemy import create_engine, text, inspect
from sqlalchemy.engine import Engine
from app.config import get_settings

class PostgresConnector:
    kind = "postgres"

    def __init__(self, dsn: str):
        s = get_settings()
        self.engine: Engine = create_engine(
            dsn,
            pool_pre_ping=True,
            pool_size=2,
            max_overflow=2,
            connect_args={"options": f"-c statement_timeout={s.statement_timeout_ms} "
                                   f"-c default_transaction_read_only=on"},
        )

    def describe_schema(self, sample_rows: int = 3) -> dict[str, Any]:
        insp = inspect(self.engine)
        out: dict[str, Any] = {"tables": []}
        with self.engine.connect() as conn:
            for table in insp.get_table_names(schema="public"):
                cols = [{"name": c["name"], "type": str(c["type"])} for c in insp.get_columns(table)]
                sample = conn.execute(text(f'SELECT * FROM "{table}" LIMIT {sample_rows}')).fetchall()
                out["tables"].append({"name": table, "columns": cols,
                                       "sample": [list(map(str, r)) for r in sample]})

        return out

    def run_select(self, sql: str, max_rows: int) -> tuple[list[str], list[tuple]]:
        with self.engine.connect() as conn:
            res = conn.execute(text(sql))
            cols = list(res.keys())
            rows = res.fetchmany(max_rows)
            return cols, [tuple(r) for r in rows]

    def test(self) -> bool:
        with self.engine.connect() as conn:
            return conn.execute(text("SELECT 1")).scalar() == 1

```

Checkpoint:

```

python - <<'EOF'
from app.connectors.postgres import PostgresConnector
c = PostgresConnector("postgresql+psycopg://analyst_ro:ro@localhost:5435/demo")
print(c.test(), [t["name"] for t in c.describe_schema()["tables"]])
EOF

```

Expected: True ['batteries', 'dealers', 'telemetry'].

6. API development

6.1 Authentication and tenant context

The Next.js app authenticates the user (Supabase) and mints a short-lived JWT for this service containing `tenant_id` and `user_id`, signed with the shared `JWT_SECRET`. This service **only** trusts that token. It never looks up users.

File: `app/security/auth.py`

```
from dataclasses import dataclass
from jose import jwt, JWTError
from app.config import get_settings

@dataclass(frozen=True)
class TenantContext:
    tenant_id: str
    user_id: str

def decode_token(token: str) -> TenantContext:
    s = get_settings()
    try:
        payload = jwt.decode(token, s.jwt_secret.get_secret_value(), algorithms=[s.jwt_algorithm])
    except JWTError as e:
        raise ValueError(f"invalid token: {e}") from e
    if "tenant_id" not in payload or "sub" not in payload:
        raise ValueError("token missing tenant_id or sub")
    return TenantContext(tenant_id=payload["tenant_id"], user_id=payload["sub"])
```

File: `app/api/deps.py`

```
import structlog
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from app.security.auth import decode_token, TenantContext

_bearer = HTTPBearer(auto_error=False)

def current_tenant(creds: HTTPAuthorizationCredentials | None = Depends(_bearer)) -> TenantContext:
    if creds is None:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "missing bearer token")
    try:
        ctx = decode_token(creds.credentials)
    except ValueError as e:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, str(e))
    structlog.contextvars.bind_contextvars(tenant_id=ctx.tenant_id, user_id=ctx.user_id)
    return ctx
```

Mint a test token for local development:

```
python - <<'EOF'
from jose import jwt
from app.config import get_settings
tok = jwt.encode({"tenant_id": "t_demo", "sub": "u_apoorv"},
                 get_settings().jwt_secret.get_secret_value(), algorithm="HS256")
print(tok)
EOF
```

Keep the output in a shell variable: `export TOKEN=<paste>`.

6.2 Credential vault

File: `app/security/vault.py`

```
import json
from cryptography.fernet import Fernet, InvalidToken
from app.config import get_settings

_f = Fernet(get_settings().credential_encryption_key.get_secret_value().encode())

def encrypt(obj: dict) -> str:
    return _f.encrypt(json.dumps(obj).encode()).decode()

def decrypt(token: str) -> dict:
    try:
        return json.loads(_f.decrypt(token.encode()))
    except InvalidToken as e:
        raise ValueError("credential decrypt failed – wrong CREDENTIAL_ENCRYPTION_KEY?" ) from e
```

Rule: `decrypt()` is called only inside `app/connectors/registry.py`. No API response ever contains its output.

6.3 Request / response schemas

File: `app/api/schemas.py`

```
from typing import Literal
from pydantic import BaseModel, Field, field_validator

class ConnectionCreate(BaseModel):
    name: str = Field(min_length=1, max_length=200)
    kind: Literal["postgres", "web"]
    # postgres: {"dsn": "postgresql://user:pass@host:5432/db"}
    # web:      {"url": "...", "username": "...", "password": "..."}
    secret: dict

    @field_validator("secret")
    @classmethod
    def _check_secret(cls, v: dict, info):
        kind = info.data.get("kind")
        need = {"postgres": {"dsn"}, "web": {"url", "username", "password"}}[kind]
        missing = need - v.keys()
        if missing:
            raise ValueError(f"secret missing {sorted(missing)}")
        return v

class ConnectionOut(BaseModel):
    id: str
    name: str
    kind: str
    has_schema_cache: bool

class RunCreate(BaseModel):
    connection_id: str
    thread_id: str = Field(min_length=1, max_length=100)
    question: str = Field(min_length=3, max_length=2000)
```

Note `ConnectionOut` has no `secret` field. This is deliberate and non-negotiable.

6.4 Connections service and routes

File: `app/services/errors.py`

```
class DomainError(Exception):
    status_code = 400

class NotFound(DomainError):
    status_code = 404

class Forbidden(DomainError):
    status_code = 403

class GuardRejected(DomainError):
    """SQL failed the safety guard."""
    status_code = 422

class BudgetExceeded(DomainError):
    status_code = 429
```

File: `app/services/connections.py`

```

from sqlalchemy.orm import Session
from app.db.models import Connection, Tenant
from app.security import vault
from app.connectors.postgres import PostgresConnector
from app.services.errors import NotFound

def ensure_tenant(db: Session, tenant_id: str) -> Tenant:
    t = db.get(Tenant, tenant_id)
    if t is None:
        t = Tenant(id=tenant_id, name=tenant_id)
        db.add(t); db.commit()
    return t

def create_connection(db: Session, tenant_id: str, name: str, kind: str, secret: dict) -> Connection:
    ensure_tenant(db, tenant_id)
    if kind == "postgres":
        PostgresConnector(secret["dsn"]).test() # fail fast before storing
    conn = Connection(tenant_id=tenant_id, name=name, kind=kind, secret_enc=vault.encrypt(secret))
    db.add(conn); db.commit(); db.refresh(conn)
    return conn

def list_connections(db: Session, tenant_id: str) -> list[Connection]:
    return db.query(Connection).filter(Connection.tenant_id == tenant_id).all()

def get_connection(db: Session, tenant_id: str, connection_id: str) -> Connection:
    conn = db.get(Connection, connection_id)
    if conn is None or conn.tenant_id != tenant_id:
        raise NotFound("connection not found") # 404 for both: do not leak existence
    return conn

```

File: `app/connectors/registry.py`

```

from app.db.models import Connection
from app.security import vault
from app.connectors.postgres import PostgresConnector

def connector_for(conn: Connection):
    secret = vault.decrypt(conn.secret_enc)
    if conn.kind == "postgres":
        return PostgresConnector(secret["dsn"])
    raise ValueError(f"unsupported connection kind {conn.kind}")

```

File: `app/api/routes_connections.py`

```

from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from app.api.deps import current_tenant
from app.api.schemas import ConnectionCreate, ConnectionOut
from app.db.session import get_db
from app.security.auth import TenantContext
from app.services import connections as svc

router = APIRouter()

def _out(c) -> ConnectionOut:
    return ConnectionOut(id=c.id, name=c.name, kind=c.kind, has_schema_cache=c.schema_cache is not None)

@router.post("", response_model=ConnectionOut, status_code=201)
def create(body: ConnectionCreate, ctx: TenantContext = Depends(current_tenant),
          db: Session = Depends(get_db)):
    return _out(svc.create_connection(db, ctx.tenant_id, body.name, body.kind, body.secret))

@router.get("", response_model=list[ConnectionOut])
def list_(ctx: TenantContext = Depends(current_tenant), db: Session = Depends(get_db)):
    return [_out(c) for c in svc.list_connections(db, ctx.tenant_id)]

```

Register the domain-error handler in `app/main.py` (add inside `create_app`, before `return app`):

```

from fastapi import Request
from fastapi.responses import JSONResponse
from app.services.errors import DomainError

@app.exception_handler(DomainError)
async def _domain_error(_: Request, exc: DomainError):
    return JSONResponse(status_code=exc.status_code, content={"error": str(exc)})

```

Checkpoint:

```

curl -s -X POST localhost:8000/connections \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"name": "demo", "kind": "postgres", "secret": {"dsn": "postgresql+psycopg://analyst_ro:ro@host.docker.internal:5435/demo"}}'

```

(Use `localhost` instead of `host.docker.internal` when uvicorn runs on your host.)

Expected: `{"id": "<uuid>", "name": "demo", "kind": "postgres", "has_schema_cache": false}` with status 201.

`curl -s localhost:8000/connections -H "Authorization: Bearer $TOKEN"` lists it. Without the header you get 401.

Save the id: `export CONN=<uuid>`.

6.5 Runs route (SSE)

This route creates a `Run` row, streams graph events to the client, and finalises the row with usage. The graph itself is built in section 7.

File: `app/api/routes_runs.py`

```
import json
from fastapi import APIRouter, Depends
from sse_starlette.sse import EventSourceResponse
from sqlalchemy.orm import Session
from app.api.deps import current_tenant
from app.api.schemas import RunCreate
from app.db.session import get_db
from app.security.auth import TenantContext
from app.services import runs as svc

router = APIRouter()

@router.post("")
async def create_run(body: RunCreate, ctx: TenantContext = Depends(current_tenant),
                    db: Session = Depends(get_db)):
    async def event_stream():
        async for event in svc.stream_run(db, ctx, body):
            yield {"event": event["type"], "data": json.dumps(event["data"])}
    return EventSourceResponse(event_stream())
```

Event contract (this is the OpenAPI-adjacent contract the Next.js client is generated from):

event	data	when
status	{"stage": "router" \ "sql_gen" \ "sql_guard" \ "db_exec" \ "answer"}	entering each node
sql	{"sql": "SELECT ..."}	after the guard accepts a query
rows	{"columns": [...], "rows": [[...]], "truncated": bool}	after execution
token	{"text": "..."}	each streamed answer chunk
done	{"run_id": "...", "duration_ms": 1234}	end
error	{"message": "..."}	any failure; stream closes after

7. Business logic — the LangGraph agent

7.1 State

File: `app/agent/state.py`

```
from typing import TypedDict, Literal, Any

class AgentState(TypedDict, total=False):
    # inputs
    tenant_id: str
    connection_id: str
    question: str
    schema: dict[str, Any] # from connector.describe_schema()
    # routing
    tool: Literal["sql", "web", "clarify"]
    # sql path
    sql: str
    guard_error: str | None
    retries: int
    columns: list[str]
    rows: list[list[Any]]
    truncated: bool
    # output
    answer: str
    error: str | None
```

`total=False` lets nodes return partial updates; LangGraph merges them.

7.2 Prompts

File: `app/agent/prompts.py`

```
ROUTER_SYSTEM = """You classify a business question about a customer's data.
Return `sql` if it can be answered from the database tables listed.
Return `web` only if the question needs LIVE data from the customer's web dashboard
(words like 'right now', 'current', 'live', 'at this moment').
Return `clarify` if the question is not about the data at all."""

SQL_SYSTEM = """You write ONE PostgreSQL SELECT statement to answer the question.
Rules:
- Use only the tables and columns in the schema below. Never invent columns.
- Never write INSERT, UPDATE, DELETE, DROP, ALTER, or CTEs that modify data.
- Prefer explicit column lists over SELECT *.
- Aggregate when the question asks for totals, counts, averages, or rankings.
- Add ORDER BY and LIMIT for 'top N' questions.
- Dates: the current date is {today}. 'last month' means the previous calendar month.
Return only the SQL, no prose, no code fences."""

ANSWER_SYSTEM = """You are a concise analyst. Given the question, the SQL, and the result rows,
write a 2-4 sentence answer in plain English with the key numbers.
If the result is empty, say so plainly and suggest one reason.
Do not mention that you are an AI. Do not repeat the SQL."""
```

7.3 Router node

File: `app/agent/nodes/router.py`

```
from typing import Literal
from pydantic import BaseModel
from langchain_core.messages import SystemMessage, HumanMessage
from app.llm import get_llm
from app.agent.prompts import ROUTER_SYSTEM
from app.agent.state import AgentState

class RouteDecision(BaseModel):
    tool: Literal["sql", "web", "clarify"]
    reason: str

def _schema_summary(schema: dict) -> str:
    return "\n".join(f"- {t['name']}" + (f'({', '.join(c['name'] for c in t['columns'])})'
    for t in schema.get("tables", []))

def router_node(state: AgentState) -> AgentState:
    llm = get_llm().with_structured_output(RouteDecision)
    decision: RouteDecision = llm.invoke([
        SystemMessage(content=ROUTER_SYSTEM),
        HumanMessage(content=f"Tables:\n{_schema_summary(state['schema'])}\n\nQuestion: {state['question']}"),
    ])
    return {"tool": decision.tool}
```

7.4 SQL generation node

File: `app/agent/nodes/sql_gen.py`

```
import json
from datetime import date
from langchain_core.messages import SystemMessage, HumanMessage
from app.llm import get_llm
from app.agent.prompts import SQL_SYSTEM
from app.agent.state import AgentState

def _schema_ddl(schema: dict) -> str:
    parts = []
    for t in schema["tables"]:
        cols = ", ".join(f"{c['name']} {c['type']}" for c in t["columns"])
        parts.append(f"TABLE {t['name']} ({cols})\n sample rows: {json.dumps(t['sample'][:3])}")
    return "\n".join(parts)

def sql_gen_node(state: AgentState) -> AgentState:
    msgs = [
        SystemMessage(content=SQL_SYSTEM.format(today=date.today().isoformat())),
        HumanMessage(content=f"Schema:\n{_schema_ddl(state['schema'])}\n\nQuestion: {state['question']}"),
    ]
    if state.get("guard_error"):
        msgs.append(HumanMessage(content=f"Your previous SQL was rejected:\n{state['sql']}\n"
                                     f"Reason: {state['guard_error']}\nWrite a corrected query."))
    sql = get_llm().invoke(msgs).content.strip().strip("`")
    if sql.lower().startswith("sql"):
        sql = sql[3:].strip()
    return {"sql": sql, "guard_error": None}
```

7.5 SQL guard node — code, not LLM

This is the most important file in the service. It must never call a model.

File: `app/agent/nodes/sql_guard.py`

```
import sqlglot
from sqlglot import exp
from app.agent.state import AgentState
from app.config import get_settings

FORBIDDEN = (exp.Insert, exp.Update, exp.Delete, exp.Drop, exp.Alter, exp.Create,
              exp.Command, exp.Merge, exp.TruncateTable, exp.Grant)

def validate_sql(sql: str, allowed_tables: set[str], max_rows: int) -> tuple[str, str | None]:
    """Return (safe_sql, None) or (original_sql, error)."""
    try:
        statements = sqlglot.parse(sql, read="postgres")
    except sqlglot.errors.ParseError as e:
        return sql, f"parse error: {e}"
    if len(statements) != 1:
        return sql, "exactly one statement is allowed"
    tree = statements[0]
    if not isinstance(tree, exp.Select) and not isinstance(tree, exp.Union):
        return sql, "only SELECT statements are allowed"
    for node in tree.walk():
        if isinstance(node, FORBIDDEN):
            return sql, f"forbidden operation: {type(node).__name__}"
    used = {t.name.lower() for t in tree.find_all(exp.Table)}
    unknown = used - {t.lower() for t in allowed_tables}
    if unknown:
        return sql, f"tables not allowed: {sorted(unknown)}"
    # enforce a row cap
    limit = tree.args.get("limit")
    if limit is None or int(limit.expression.this) > max_rows:
        tree = tree.limit(max_rows)
    return tree.sql(dialect="postgres"), None

def sql_guard_node(state: AgentState) -> AgentState:
    allowed = {t["name"] for t in state["schema"]["tables"]}
    safe_sql, err = validate_sql(state["sql"], allowed, get_settings().max_rows)
    if err:
        return {"guard_error": err, "retries": state.get("retries", 0) + 1}
    return {"sql": safe_sql, "guard_error": None}
```

7.6 Execution and answer nodes

File: `app/agent/nodes/db_exec.py`

```

from app.agent.state import AgentState
from app.config import get_settings

def make_db_exec_node(connector):
    """Closure so the node has the tenant's connector without it living in state."""
    def db_exec_node(state: AgentState) -> AgentState:
        max_rows = get_settings().max_rows
        try:
            cols, rows = connector.run_select(state["sql"], max_rows + 1)
        except Exception as e:
            # DB error -> feed back to sql_gen
            return {"guard_error": f"database error: {e}", "retries": state.get("retries", 0) + 1}
        truncated = len(rows) > max_rows
        return {"columns": cols, "rows": [list(r) for r in rows[:max_rows]],
            "truncated": truncated, "guard_error": None}
    return db_exec_node

```

File: app/agent/nodes/answer.py

```

import json
from langchain_core.messages import SystemMessage, HumanMessage
from app.llm import get_llm
from app.agent.prompts import ANSWER_SYSTEM
from app.agent.state import AgentState

def answer_node(state: AgentState) -> AgentState:
    preview = {"columns": state.get("columns", []), "rows": state.get("rows", [])[:50]}
    resp = get_llm(temperature=0.2).invoke([
        SystemMessage(content=ANSWER_SYSTEM),
        HumanMessage(content=f"Question: {state['question']}\nSQL: {state.get('sql')}\n"
            f"Result: {json.dumps(preview, default=str)}"),
    ])
    return {"answer": resp.content}

```

7.7 Web tool node (Playwright)

Phase 3. Included here so the graph shape is complete; wire it in once the SQL path passes evals.

File: app/workers/web_session.py

```

import json, os
from pathlib import Path
from playwright.sync_api import sync_playwright
from app.config import get_settings

def _state_path(tenant_id: str, connection_id: str) -> Path:
    p = Path(get_settings().session_store_dir) / tenant_id
    p.mkdir(parents=True, exist_ok=True)
    return p / f"{connection_id}.json"

def fetch_dashboard_json(tenant_id: str, connection_id: str, secret: dict,
    data_url_substring: str) -> list[dict]:
    """Log in (or reuse a saved session), open the dashboard, capture the XHR that
    contains `data_url_substring`, return its JSON payload."""
    s = get_settings()
    state = _state_path(tenant_id, connection_id)
    captured: list[dict] = []
    with sync_playwright() as pw:
        browser = pw.chromium.launch(headless=s.playwright_headless)
        ctx = browser.new_context(storage_state=str(state) if state.exists() else None)
        page = ctx.new_page()
        page.on("response", lambda r: captured.append(r.json())
            if data_url_substring in r.url and "json" in (r.headers.get("content-type") or "") else None)
        page.goto(secret["url"], wait_until="networkidle")
        if page.locator("input[type=password]").count():
            # session expired -> login
            page.fill("input[type=text]", input[type=email], secret["username"])
            page.fill("input[type=password]", secret["password"])
            page.keyboard.press("Enter")
            page.wait_for_load_state("networkidle")
            ctx.storage_state(path=str(state))
        page.wait_for_timeout(1500)
        browser.close()
    if not captured:
        raise RuntimeError("no dashboard data captured - check data_url_substring")
    return captured[-1] if isinstance(captured[-1], list) else [captured[-1]]

```

File: app/agent/nodes/web_tool.py

```

from app.agent.state import AgentState

def make_web_tool_node(tenant_id: str, connection_id: str, secret: dict):
    def web_tool_node(state: AgentState) -> AgentState:
        from app.workers.web_session import fetch_dashboard_json
        try:
            data = fetch_dashboard_json(tenant_id, connection_id, secret,
                                       data_url_substring=secret.get("data_url_match", "/api/"))

        except Exception as e:
            return {"error": f"web tool failed: {e}"}
        cols = sorted({k for row in data for k in row.keys()})
        rows = [[row.get(c) for c in cols] for row in data]
        return {"columns": cols, "rows": rows, "truncated": False, "sql": None}
    return web_tool_node

```

Playwright's sync API blocks the event loop; run it via `await asyncio.to_thread(...)` from the service layer, and move it to a Redis worker in phase 4.

7.8 Building the graph

File: `app/agent/graph.py`

```

from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.postgres import PostgresSaver
from app.agent.state import AgentState
from app.agent.nodes.router import router_node
from app.agent.nodes.sql_gen import sql_gen_node
from app.agent.nodes.sql_guard import sql_guard_node
from app.agent.nodes.db_exec import make_db_exec_node
from app.agent.nodes.answer import answer_node
from app.config import get_settings

def _after_router(state: AgentState) -> str:
    return {"sql": "sql_gen", "web": "web_tool", "clarify": "answer"}[state["tool"]]

def _after_guard(state: AgentState) -> str:
    if state.get("guard_error") is None:
        return "db_exec"
    return "sql_gen" if state.get("retries", 0) <= get_settings().max_sql_retries else "answer"

def _after_exec(state: AgentState) -> str:
    if state.get("guard_error") is None:
        return "answer"
    return "sql_gen" if state.get("retries", 0) <= get_settings().max_sql_retries else "answer"

def build_graph(connector, web_tool_node=None, checkpointer: PostgresSaver | None = None):
    g = StateGraph(AgentState)
    g.add_node("router", router_node)
    g.add_node("sql_gen", sql_gen_node)
    g.add_node("sql_guard", sql_guard_node)
    g.add_node("db_exec", make_db_exec_node(connector))
    g.add_node("answer", answer_node)
    g.add_node("web_tool", web_tool_node or (lambda s: {"error": "web tool not configured"}))

    g.add_edge(START, "router")
    g.add_conditional_edges("router", _after_router)
    g.add_edge("sql_gen", "sql_guard")
    g.add_conditional_edges("sql_guard", _after_guard)
    g.add_conditional_edges("db_exec", _after_exec)
    g.add_edge("web_tool", "answer")
    g.add_edge("answer", END)
    return g.compile(checkpointer=checkpointer)

```

7.9 Run service — orchestration and usage

File: `app/services/runs.py`

```

import time
from datetime import datetime, timezone, timedelta
from typing import AsyncIterator
from sqlalchemy import func
from sqlalchemy.orm import Session
from langgraph.checkpoint.postgres import PostgresSaver
from langchain_core.callbacks import UsageMetadataCallbackHandler

from app.api.schemas import RunCreate
from app.config import get_settings
from app.connectors.registry import connector_for
from app.db.models import Run, Tenant
from app.security.auth import TenantContext
from app.services import connections as conn_svc
from app.services.errors import BudgetExceeded
from app.agent.graph import build_graph
from app.logging import log

STAGES = {"router", "sql_gen", "sql_guard", "db_exec", "web_tool", "answer"}

def _tokens_today(db: Session, tenant_id: str) -> int:
    since = datetime.now(timezone.utc) - timedelta(days=1)
    return db.query(func.coalesce(func.sum(Run.prompt_tokens + Run.completion_tokens), 0)) \
        .filter(Run.tenant_id == tenant_id, Run.created_at >= since).scalar()

async def stream_run(db: Session, ctx: TenantContext, body: RunCreate) -> AsyncIterator[dict]:
    s = get_settings()
    tenant: Tenant = conn_svc.ensure_tenant(db, ctx.tenant_id)
    if _tokens_today(db, tenant.id) >= tenant.daily_token_budget:
        raise BudgetExceeded("daily token budget exhausted")

    conn = conn_svc.get_connection(db, ctx.tenant_id, body.connection_id)
    connector = connector_for(conn)

    if conn.schema_cache is None or (datetime.now(timezone.utc) - conn.schema_cached_at) > timedelta(hours=6):
        conn.schema_cache = connector.describe_schema()
        conn.schema_cached_at = datetime.now(timezone.utc)
        db.commit()

    run = Run(tenant_id=ctx.tenant_id, connection_id=conn.id, thread_id=body.thread_id,
              question=body.question)
    db.add(run); db.commit(); db.refresh(run)
    log.info("run.start", run_id=run.id, connection_id=conn.id)

    usage = UsageMetadataCallbackHandler()
    t0 = time.perf_counter()
    final: dict = {}
    try:
        with PostgresSaver.from_conn_string(str(s.checkpoint_db_url)) as saver:
            graph = build_graph(connector, checkpoint=saver)
            config = {"configurable": {"thread_id": f"{ctx.tenant_id}:{body.thread_id}"},
                     "callbacks": [usage],
                     "metadata": {"tenant_id": ctx.tenant_id, "run_id": run.id}}
            inputs = {"tenant_id": ctx.tenant_id, "connection_id": conn.id,
                     "question": body.question, "schema": conn.schema_cache, "retries": 0}
            for event in graph.stream(inputs, config=config, stream_mode="updates"):
                node, update = next(iter(event.items()))
                if node in STAGES:
                    yield {"type": "status", "data": {"stage": node}}
                    if node == "sql_guard" and update.get("guard_error") is None:
                        yield {"type": "sql", "data": {"sql": update["sql"]}}
                    if node in ("db_exec", "web_tool") and "rows" in update:
                        yield {"type": "rows", "data": {"columns": update["columns"], "rows": update["rows"],
                                                       "truncated": update["truncated"]}}

                    if node == "answer":
                        yield {"type": "token", "data": {"text": update["answer"]}}
                    final.update(update)
                run.status = "error" if final.get("error") else "done"
            except Exception as e:
                log.exception("run.failed", run_id=run.id)
                run.status, run.error = "error", str(e)
                yield {"type": "error", "data": {"message": "run failed; see logs"}}
            finally:
                run.duration_ms = int((time.perf_counter() - t0) * 1000)
                run.tool, run.sql = final.get("tool"), final.get("sql")
                run.rows_returned = len(final.get("rows", []))
                totals = usage.usage_metadata.get(s.openrouter_model, {})
                run.prompt_tokens = totals.get("input_tokens", 0)
                run.completion_tokens = totals.get("output_tokens", 0)
                db.commit()
                log.info("run.end", run_id=run.id, status=run.status, ms=run.duration_ms)
            if run.status == "done":
                yield {"type": "done", "data": {"run_id": run.id, "duration_ms": run.duration_ms}}

```

`graph.stream` is synchronous and will block the event loop under load. For phase 0-2 this is acceptable. In phase 4 the same function body moves into an `arg` worker and the route reads events from a Redis stream.

7.10 End-to-end checkpoint

Restart uvicorn, then:

```

curl -N -X POST localhost:8000/runs \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d "{\"connection_id\": \"${CONN}\", \"thread_id\": \"t1\", \"question\": \"Which dealer sold the most batteries in August 2026?\"}"

```

Expected stream (abridged):

```
event: status
data: {"stage": "router"}

event: status
data: {"stage": "sql_gen"}

event: sql
data: {"sql": "SELECT d.name, COUNT(*) AS batteries_sold FROM batteries AS b JOIN dealers AS d ON ... LIMIT 500"}

event: rows
data: {"columns": ["name", "batteries_sold"], "rows": [{"Patil EV", 1}, {"Roy Auto", 1}, {"Sharma Motors", 1}], "truncated": false}

event: token
data: {"text": "Three dealers each sold one battery in August 2026 ..."}

event: done
data: {"run_id": "...", "duration_ms": 4120}
```

Verify:

1. `psql ...5433/analyst -c "select status, tool, prompt_tokens, rows_returned from runs order by created_at desc limit 1"` shows `done | sql | >0 | 3`.
2. The LangSmith project shows one trace with the router, sql_gen and answer LLM calls and metadata `tenant_id`.
3. Ask a follow-up on the same `thread_id` ("and in September?"). The checkpointeer supplies the prior turn.

Common errors

Symptom	Fix
event: error immediately, log says <code>credential decrypt failed</code>	<code>CREDENTIAL_ENCRYPTION_KEY</code> changed after the connection was stored; delete and recreate the connection
Guard rejects everything with <code>tables not allowed</code>	Schema cache is empty — the read-only role lacks <code>USAGE ON SCHEMA public</code>
429 on every run	<code>daily_token_budget</code> on the tenant row is 0 or usage rows are inflated; check <code>runs</code>
Answer says "no data" but rows are present	The model's Result: JSON is being truncated — lower <code>preview</code> to 20 rows or raise <code>max_tokens</code>

7.11 Golden evals

File: `evals/golden_sql.yaml`

```
- q: How many dealers do we have?
  expect_value: 3
- q: Total battery revenue in August 2026
  expect_value: 157300
- q: Which city has the most batteries sold?
  expect_contains: Nashik
- q: Average temperature in telemetry in the last 24 hours
  expect_type: number
```

File: `evals/run_evals.py`

```
import sys, yaml, httpx, json, os

BASE = os.environ.get("AGENT_URL", "http://localhost:8000")
TOKEN, CONN = os.environ["TOKEN"], os.environ["CONN"]
cases = yaml.safe_load(open("evals/golden_sql.yaml"))
passed = 0
for i, c in enumerate(cases):
    rows, answer = [], ""
    with httpx.stream("POST", f"{BASE}/runs", headers={"Authorization": f"Bearer {TOKEN}"},
                     json={"connection_id": CONN, "thread_id": f"eval-{i}", "question": c["q"]},
                     timeout=120) as r:
        for line in r.iter_lines():
            if line.startswith("data:"):
                d = json.loads(line[5:])
                rows = d.get("rows", rows); answer = d.get("text", answer)
    flat = [x for row in rows for x in row]
    ok = ("expect_value" in c and any(str(c["expect_value"]) in str(x) for x in flat)) \
        or ("expect_contains" in c and c["expect_contains"] in (answer + json.dumps(rows))) \
        or ("expect_type" in c and c["expect_type"] == "number" and any(isinstance(x, (int, float)) for x in flat))
    passed += ok
    print(("PASS" if ok else "FAIL"), "|", c["q"])
print(f"{passed}/{len(cases)}")
sys.exit(0 if passed / len(cases) >= 0.8 else 1)
```

Run: `python evals/run_evals.py`. Phase 1 "done" is 80%+ on 30 cases. Add a case every time a real user gets a wrong answer.

8. Testing

8.1 Configuration

pytest options live in `pyproject.toml` (section 2.3). Coverage target: **85% on `app/agent/nodes/sql_guard.py` and `app/security/`, 70% overall**. The guard and vault are where bugs become breaches.

File: `tests/conftest.py`

```
import os, pytest
from cryptography.fernet import Fernet

os.environ.setdefault("OPENROUTER_API_KEY", "test")
os.environ.setdefault("CREDENTIAL_ENCRYPTION_KEY", Fernet.generate_key().decode())
os.environ.setdefault("JWT_SECRET", "test-secret-test-secret-test-secret")
os.environ.setdefault("APP_DB_URL", "postgresql+psycpg://app:app@localhost:5433/analyst")
os.environ.setdefault("CHECKPOINT_DB_URL", "postgresql://ckpt:ckpt@localhost:5434/checkpoints")
os.environ.setdefault("LANGSMITH_TRACING", "false")

@pytest.fixture
def token():
    from jose import jwt
    from app.config import get_settings
    return jwt.encode({"tenant_id": "t_test", "sub": "u_test"},
                     get_settings().jwt_secret.get_secret_value(), algorithm="HS256")

@pytest.fixture
def client():
    from fastapi.testclient import TestClient
    from app.main import app
    return TestClient(app)
```

8.2 Unit tests

File: `tests/unit/test_sql_guard.py`

```
import pytest
from app.agent.nodes.sql_guard import validate_sql

ALLOWED = {"dealers", "batteries"}

def test_accepts_select_and_adds_limit():
    sql, err = validate_sql("select name from dealers", ALLOWED, 100)
    assert err is None and "LIMIT 100" in sql

def test_caps_oversized_limit():
    sql, err = validate_sql("select name from dealers limit 9999", ALLOWED, 100)
    assert err is None and "LIMIT 100" in sql

@pytest.mark.parametrize("bad", [
    "delete from dealers",
    "update dealers set name='x'",
    "drop table dealers",
    "select 1; delete from dealers",
    "select * from secrets",
    "with x as (delete from dealers returning *) select * from x",
])
def test_rejects(bad):
    _, err = validate_sql(bad, ALLOWED, 100)
    assert err is not None
```

File: `tests/unit/test_vault.py`

```
from app.security import vault

def test_roundtrip():
    assert vault.decrypt(vault.encrypt({"dsn": "x"})) == {"dsn": "x"}

def test_ciphertext_hides_plaintext():
    assert "dsn" not in vault.encrypt({"dsn": "postgresql://u:p@h/db"})
```

File: `tests/unit/test_router.py` — mock the LLM so unit tests never hit the network:

```
from unittest.mock import patch
from app.agent.nodes.router import router_node, RouteDecision

def test_router_returns_tool():
    class FakeLLM:
        def with_structured_output(self, _): return self
        def invoke(self, _): return RouteDecision(tool="sql", reason="tables cover it")
    with patch("app.agent.nodes.router.get_llm", return_value=FakeLLM()):
        out = router_node({"question": "how many dealers", "schema": {"tables": []}})
    assert out == {"tool": "sql"}
```

8.3 Integration tests

These need the Compose databases running. Mark them so CI can run them in a separate job.

File: tests/integration/test_connections_api.py

```
import pytest

pytestmark = pytest.mark.integration
DSN = "postgresql+psycopg://analyst_ro:ro@localhost:5435/demo"

def test_requires_auth(client):
    assert client.get("/connections").status_code == 401

def test_create_and_list(client, token):
    h = {"Authorization": f"Bearer {token}"}
    r = client.post("/connections", headers=h,
                  json={"name": "demo", "kind": "postgres", "secret": {"dsn": DSN}})
    assert r.status_code == 201
    body = r.json()
    assert "secret" not in body and "dsn" not in str(body)
    assert any(c["id"] == body["id"] for c in client.get("/connections", headers=h).json())

def test_other_tenant_cannot_see_it(client, token):
    from jose import jwt
    from app.config import get_settings
    other = jwt.encode({"tenant_id": "t_other", "sub": "u"},
                      get_settings().jwt_secret.get_secret_value(), algorithm="HS256")
    assert client.get("/connections", headers={"Authorization": f"Bearer {other}"}) == []
```

Add to pyproject.toml under [tool.pytest.ini_options]:

```
markers = ["integration: needs docker compose databases"]
```

Run:

```
pytest -m "not integration"           # fast, no network, no DB
pytest -m integration                 # needs docker compose up
pytest --cov=app --cov-report=html    # open htmlcov/index.html
```

Expected: all green; coverage report lists sql_guard.py at 90%+.

Common errors

Symptom	Fix
Unit tests try to open Postgres	app/db/session.py creates the engine at import; wrap in a function or accept that create_engine is lazy (it is — no connection is opened until used)
RuntimeError: There is no current event loop	Add asyncio_mode = "auto" (already in pyproject)
Integration tests pass locally, fail in CI	CI needs the services: block with Postgres containers (section 9.5)

9. Deployment preparation

9.1 Environment variables in production

- Never bake secrets into the image. Pass them with `--env-file` or the host's secret store.
- `CREDENTIAL_ENCRYPTION_KEY` rotation: generate a new key, decrypt every `connections.secret_enc` with the old key and re-encrypt with the new, in one transaction. Losing this key means every tenant re-enters credentials.
- `APP_ENV=prod` switches logs to JSON and must also switch `allow_origins` to your real domain.

Production `.env` differences from dev:

```
APP_ENV=prod
LOG_LEVEL=INFO
APP_DB_URL=postgresql+psycopg://app:<strong>app-db:5432/analyst
CHECKPOINT_DB_URL=postgresql://ckpt:<strong>checkpoint-db:5432/checkpoints
PLAYWRIGHT_HEADLESS=true
LANGSMITH_PROJECT=analyst-agent-prod
```

9.2 Dockerfile

Use the official Playwright image so Chromium and its system libraries are already present.

File: `Dockerfile`

```
FROM mcr.microsoft.com/playwright/python:v1.47.0-jammy

ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
WORKDIR /srv

COPY pyproject.toml requirements.lock ./
RUN pip install --upgrade pip && pip install -r requirements.lock

COPY app ./app
COPY alembic ./alembic
COPY alembic.ini ./
RUN pip install --no-deps -e .

RUN useradd -m agent && mkdir -p /srv/sessions && chown -R agent /srv
USER agent

EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s CMD curl -fs http://localhost:8000/health || exit 1
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

Build and smoke-test:

```
docker build -t analyst-agent:local .
docker run --rm --env-file .env -p 8000:8000 --network host analyst-agent:local &
sleep 3 && curl -s localhost:8000/health
```

9.3 Production Compose

File: `docker-compose.yml`

```

services:
  agent:
    image: ghcr.io/itarangit/analyst-agent:${TAG:-latest}
    env_file: .env
    depends_on: [app-db, checkpoint-db, redis]
    volumes: ["sessions:/srv/sessions"]
    restart: unless-stopped
    command: >
      sh -c "alembic upgrade head &&
        uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 2"

  app-db:
    image: postgres:16
    env_file: .env.db
    volumes: ["app-db:/var/lib/postgresql/data"]
    restart: unless-stopped

  checkpoint-db:
    image: postgres:16
    env_file: .env.ckpt
    volumes: ["ckpt-db:/var/lib/postgresql/data"]
    restart: unless-stopped

  redis:
    image: redis:7
    restart: unless-stopped

  caddy:
    image: caddy:2
    ports: ["80:80", "443:443"]
    volumes: ["/Caddyfile:/etc/caddy/Caddyfile:ro", "caddy:/data"]
    restart: unless-stopped

volumes: { app-db: {}, ckpt-db: {}, sessions: {}, caddy: {} }

```

File: Caddyfile

```

agent.yourdomain.com {
  reverse_proxy agent:8000 {
    flush_interval -1    # required for SSE
  }
}

```

`flush_interval -1` is the line people forget; without it Caddy buffers the SSE stream and the chat UI shows nothing until the run ends.

9.4 Logging in production

Structured JSON logs go to stdout; Docker collects them. Every line carries `tenant_id`, `run_id`, `level`, `timestamp`. To tail: `docker compose logs -f agent | jq`. Ship to a hosted sink (Better Stack, Grafana Loki) when you have more than one VPS.

Add Sentry (optional, recommended before first paying customer):

```
pip install sentry-sdk[fastapi]
```

```

# app/main.py, inside lifespan before yield
import sentry_sdk
if s.env == "prod" and os.getenv("SENTRY_DSN"):
    sentry_sdk.init(dsn=os.getenv("SENTRY_DSN"), traces_sample_rate=0.1)

```

9.5 CI/CD (GitHub Actions)

File: `.github/workflows/ci.yml`

```

name: ci
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      app-db:
        image: postgres:16
        env: { POSTGRES_USER: app, POSTGRES_PASSWORD: app, POSTGRES_DB: analyst }
        ports: ["5433:5432"]
      checkpoint-db:
        image: postgres:16
        env: { POSTGRES_USER: ckpt, POSTGRES_PASSWORD: ckpt, POSTGRES_DB: checkpoints }
        ports: ["5434:5432"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - with: { python-version: "3.12" }
      - run: pip install -e ".[dev]"
      - run: ruff check app tests
      - run: pytest -m "not integration"
      - run: alembic upgrade head && pytest -m integration
      env:
        OPENROUTER_API_KEY: test
        JWT_SECRET: ci-secret-ci-secret-ci-secret-ci
        CREDENTIAL_ENCRYPTION_KEY: ${ secrets.TEST_FERNET_KEY }
        APP_DB_URL: postgresql+psycpg://app:app@localhost:5433/analyst
        CHECKPOINT_DB_URL: postgresql://ckpt:ckpt@localhost:5434/checkpoints

  build:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions: { packages: write, contents: read }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
      - with: { registry: ghcr.io, username: ${ github.actor }, password: ${ secrets.GITHUB_TOKEN } }
      - uses: docker/build-push-action@v6
      with:
        push: true
        tags: ghcr.io/itarangit/analyst-agent:${ github.sha },ghcr.io/itarangit/analyst-agent:latest

```

Deploy on the VPS:

```
ssh deploy@vps 'cd /srv/analyst && TAG=<sha> docker compose pull agent && docker compose up -d agent'
```

Rollback = the same command with the previous <sha>.

9.6 Production readiness checklist

Run through every line before the first external tenant.

Security

- [] Customer DB connections use a role with `default_transaction_read_only=on` (verified by attempting a DELETE)
- [] `sql_guard` tests pass, including CTE-with-DELETE and multi-statement cases
- [] No API response contains `secret_enc`, `dsn`, or `password` (grep the OpenAPI JSON at `/openapi.json`)
- [] `JWT_SECRET` is 32+ random bytes and identical in Next.js
- [] `CREDENTIAL_ENCRYPTION_KEY` is backed up outside the VPS
- [] `CORS allow_origins` is the production domain only

Tenancy

- [] `get_connection` returns 404 for another tenant's id (integration test)
- [] `thread_id` is namespaced `tenant_id:thread_id` in the checkpointer
- [] Playwright sessions are stored under `sessions/<tenant_id>/`

Reliability

- [] `statement_timeout` set on the connector **and** on the role
- [] `max_rows` enforced by the guard and re-checked in `db_exec`
- [] Retry loop terminates (`max_sql_retries`) — test with an impossible question
- [] `/health` behind the load balancer; container `HEALTHCHECK` passing
- [] Alembic runs on container start; a failed migration stops the deploy

Observability

- [] Every run has a LangSmith trace tagged with `tenant_id` and `run_id`
- [] `runs` table records tokens, rows, duration for every run including failures
- [] Logs are JSON in prod and readable with `jq`

Operations

- [] Daily `pg_dump` of App DB to off-box storage; Checkpoint DB is disposable
- [] Rollback command documented and tested once
- [] `evals/run_evals.py` at 80%+ on the demo DB after every deploy

10. Frontend overview — Next.js + TypeScript

The frontend is a separate repository, `analyst-web`. It owns everything the customer sees and everything about *who* the customer is: sign-up, login, tenants, connections UI, chat, billing. It owns **no** customer data and **no** LLM logic — every question goes to `analyst-agent` over the `/runs` SSE endpoint defined in section 6.5.

10.1 Responsibilities split

Concern	analyst-web (Next.js)	analyst-agent (Python)
Sign-up / login	Supabase Auth	—
Tenant + membership	App tables in Supabase Postgres	reads <code>tenant_id</code> from JWT only
Agent JWT	mints HS256 token with <code>tenant_id</code> , <code>sub</code> , 15-min expiry	verifies it
Connections	form UI, calls <code>POST /connections</code>	validates, encrypts, stores
Chat	SSE client, renders events	streams events
Billing	Razorpay / Stripe checkout + webhooks, sets <code>plan</code>	reads <code>daily_token_budget</code>
Usage display	reads <code>runs</code> aggregates via a small Next.js API route	writes <code>runs</code>

Rule: the browser never talks to the Python service directly with a long-lived secret. It calls a Next.js route handler that mints the short-lived agent JWT server-side, then the browser uses that token for the SSE call.

10.2 Stack

Layer	Choice
Framework	Next.js 15 (App Router), TypeScript strict
UI	Tailwind CSS 4, shadcn/ui
Auth	Supabase Auth (<code>@supabase/ssr</code>)
Data fetching	TanStack Query 5
Forms / validation	react-hook-form + zod
API client	generated from the agent's <code>/openapi.json</code> with <code>openapi-typescript</code> + <code>openapi-fetch</code>
SSE	native <code>fetch</code> + <code>ReadableStream</code> parser (POST with body, so <code>EventSource</code> cannot be used)
Tests	Vitest + Testing Library, Playwright for e2e
Deploy	Docker on the same VPS behind Caddy, or Vercel

11. Frontend environment setup

11.1 Prerequisites

Tool	Version	Check
Node.js	20 LTS or 22 LTS	<code>node --version</code>
pnpm	9+	<code>pnpm --version</code> (corepack enable && corepack prepare pnpm@latest --activate)
Supabase project	any	URL + anon key + service role key from dashboard

11.2 Create the project

```
cd ~/code
pnpm create next-app@latest analyst-web --typescript --tailwind --eslint --app --src-dir --import-alias "@/*" --use-pnpm
cd analyst-web
pnpm add @supabase/supabase-js @supabase/ssr @tanstack/react-query jose zod react-hook-form @hookform/resolvers openapi-fetch
pnpm add -D openapi-typescript vitest @vitejs/plugin-react @testing-library/react @testing-library/jest-dom jsdom @playwright/test
pnpm dlx shadcn@latest init -d
pnpm dlx shadcn@latest add button input card table badge textarea skeleton
```

Checkpoint: `pnpm dev` and open `http://localhost:3000` — the Next.js starter page renders.

11.3 Environment variables

File: `.env.example`

```
NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...           # server only

AGENT_URL=http://localhost:8000             # server-side base URL of analyst-agent
NEXT_PUBLIC_AGENT_URL=http://localhost:8000 # browser-side (SSE)
AGENT_JWT_SECRET=change-me-32-chars-minimum # MUST equal JWT_SECRET in analyst-agent

RAZORPAY_KEY_ID=
RAZORPAY_KEY_SECRET=
RAZORPAY_WEBHOOK_SECRET=

cp .env.example .env.local
```

Validate at startup so a missing key fails fast, not on first click:

File: `src/lib/env.ts`

```
import { z } from "zod";

const server = z.object({
  SUPABASE_SERVICE_ROLE_KEY: z.string().min(20),
  AGENT_URL: z.string().url(),
  AGENT_JWT_SECRET: z.string().min(32),
});
const client = z.object({
  NEXT_PUBLIC_SUPABASE_URL: z.string().url(),
  NEXT_PUBLIC_SUPABASE_ANON_KEY: z.string().min(20),
  NEXT_PUBLIC_AGENT_URL: z.string().url(),
});

export const clientEnv = client.parse({
  NEXT_PUBLIC_SUPABASE_URL: process.env.NEXT_PUBLIC_SUPABASE_URL,
  NEXT_PUBLIC_SUPABASE_ANON_KEY: process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY,
  NEXT_PUBLIC_AGENT_URL: process.env.NEXT_PUBLIC_AGENT_URL,
});
export const serverEnv = () => server.parse(process.env); // call only in server code
```

Common errors

Symptom	Fix
ZodError on <code>NEXT_PUBLIC_*</code> at build	Next.js inlines these at build time; they must be present in the build environment, not only at runtime
Agent returns 401 for every request	<code>AGENT_JWT_SECRET</code> $\neq$ <code>JWT_SECRET</code> in the Python <code>.env</code>

12. Frontend project structure

```
analyst-web/
├── package.json
├── .env.example
├── next.config.ts
├── tailwind.config.ts
├── vitest.config.ts
├── playwright.config.ts
├── supabase/
│   └── migrations/
│       └── 0001_tenants.sql
├── src/
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── page.tsx                # marketing / redirect
│   │   ├── (auth)/
│   │   │   ├── login/page.tsx
│   │   │   └── auth/callback/route.ts # Supabase OAuth / magic-link callback
│   │   ├── (app)/
│   │   │   ├── layout.tsx          # auth guard + sidebar + QueryProvider
│   │   │   ├── chat/page.tsx
│   │   │   ├── connections/page.tsx
│   │   │   ├── usage/page.tsx
│   │   │   └── billing/page.tsx
│   │   └── api/
│   │       ├── agent-token/route.ts # mints short-lived agent JWT
│   │       ├── connections/route.ts # proxies to analyst-agent (server-side)
│   │       └── billing/webhook/route.ts
│   ├── components/
│   │   ├── ui/                    # shadcn
│   │   ├── chat/
│   │   │   ├── ChatPanel.tsx
│   │   │   ├── MessageList.tsx
│   │   │   ├── ResultTable.tsx
│   │   │   └── SqlDisclosure.tsx
│   │   └── connections/ConnectionForm.tsx
│   ├── lib/
│   │   ├── env.ts
│   │   ├── supabase/
│   │   │   ├── client.ts          # browser client
│   │   │   ├── server.ts          # server client (cookies)
│   │   │   └── middleware.ts
│   │   ├── agent/
│   │   │   ├── openapi.d.ts       # generated
│   │   │   ├── client.ts          # typed openapi-fetch client
│   │   │   ├── token.ts           # jose signing (server only)
│   │   │   ├── sse.ts             # SSE parser for POST streams
│   │   └── tenant.ts              # resolve current user -> tenant_id
│   ├── hooks/
│   │   ├── useAgentToken.ts
│   │   ├── useConnections.ts
│   │   └── useRun.ts               # streaming run state machine
│   └── middleware.ts               # refresh Supabase session on every request
├── tests/
│   ├── unit/sse.test.ts
│   ├── unit/useRun.test.tsx
│   └── e2e/chat.spec.ts
```

Why this shape

- Route handlers under `src/app/api/` are the only place that holds `SUPABASE_SERVICE_ROLE_KEY` and `AGENT_JWT_SECRET`.
- `lib/agent/` is the single boundary to the Python service. If the contract changes, only this folder and the generated types change.
- `hooks/useRun.ts` is a state machine (`idle` → `routing` → `sql` → `rows` → `answering` → `done` | `error`) driven purely by SSE events, so the UI is a rendering of agent state, not a second copy of the logic.

13. Auth, tenants and the agent token

13.1 Supabase tables

File: supabase/migrations/0001_tenants.sql

```
create table public.tenants (
  id text primary key,
  name text not null,
  plan text not null default 'free',
  created_at timestamptz default now()
);

create table public.memberships (
  user_id uuid references auth.users(id) on delete cascade,
  tenant_id text references public.tenants(id) on delete cascade,
  role text not null default 'owner',
  primary key (user_id, tenant_id)
);

alter table public.tenants enable row level security;
alter table public.memberships enable row level security;

create policy "members read own tenant" on public.tenants
  for select using (exists (select 1 from public.memberships m
    where m.tenant_id = tenants.id and m.user_id = auth.uid()));
create policy "members read own memberships" on public.memberships
  for select using (user_id = auth.uid());

-- create a tenant + membership automatically on sign-up
create or replace function public.handle_new_user() returns trigger
language plpgsql security definer as $$
declare t_id text := 't_' || replace(new.id::text, '-', '');
begin
  insert into public.tenants (id, name) values (t_id, coalesce(new.email, 'tenant'));
  insert into public.memberships (user_id, tenant_id) values (new.id, t_id);
  return new;
end $$;

create trigger on_auth_user_created after insert on auth.users
  for each row execute procedure public.handle_new_user();
```

Apply with `supabase db push` (CLI) or paste into the SQL editor. Checkpoint: sign up a test user in the Supabase dashboard → a row appears in `tenants` and `memberships`.

13.2 Supabase clients

File: src/lib/supabase/server.ts

```
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";
import { clientEnv } from "@lib/env";

export async function supabaseServer() {
  const store = await cookies();
  return createServerClient(clientEnv.NEXT_PUBLIC_SUPABASE_URL, clientEnv.NEXT_PUBLIC_SUPABASE_ANON_KEY, {
    cookies: {
      getAll: () => store.getAll(),
      setAll: (all) => all.forEach(({ name, value, options }) => store.set(name, value, options)),
    },
  });
}
```

File: src/lib/supabase/client.ts

```
import { createBrowserClient } from "@supabase/ssr";
import { clientEnv } from "@lib/env";
export const supabaseBrowser = () =>
  createBrowserClient(clientEnv.NEXT_PUBLIC_SUPABASE_URL, clientEnv.NEXT_PUBLIC_SUPABASE_ANON_KEY);
```

File: src/middleware.ts — refreshes the session cookie and protects (app) routes.

```
import { NextResponse, type NextRequest } from "next/server";
import { createServerClient } from "@supabase/ssr";

export async function middleware(req: NextRequest) {
  const res = NextResponse.next({ request: req });
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!, process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    { cookies: { getAll: () => req.cookies.getAll(), setAll: (all) => all.forEach(({ name, value, options }) => res.cookies.set(name, value, options)) } });
  const { data: { user } } = await supabase.auth.getUser();
  const protectedPath = /^\/(chat|connections|usage|billing)/.test(req.nextUrl.pathname);
  if (protectedPath && !user) return NextResponse.redirect(new URL("/login", req.url));
  return res;
}
export const config = { matcher: ["/(?!_next|favicon.ico|api|billing/webhook).*" ] };
```

13.3 Resolve the tenant

File: `src/lib/tenant.ts`

```
import { supabaseServer } from "@lib/supabase/server";

export async function currentTenant() {
  const sb = await supabaseServer();
  const { data: { user } } = await sb.auth.getUser();
  if (!user) return null;
  const { data } = await sb.from("memberships").select("tenant_id, role").eq("user_id", user.id).limit(1).single();
  if (!data) return null;
  return { userId: user.id, tenantId: data.tenant_id as string, role: data.role as string };
}
```

13.4 Mint the agent JWT (server only)

File: `src/lib/agent/token.ts`

```
import { SignJWT } from "jose";
import { serverEnv } from "@lib/env";

export async function mintAgentToken(tenantId: string, userId: string) {
  const secret = new TextEncoder().encode(serverEnv().AGENT_JWT_SECRET);
  return new SignJWT({ tenant_id: tenantId })
    .setProtectedHeader({ alg: "HS256" })
    .setSubject(userId)
    .setIssuedAt()
    .setExpirationTime("15m")
    .sign(secret);
}
```

File: `src/app/api/agent-token/route.ts`

```
import { NextResponse } from "next/server";
import { currentTenant } from "@lib/tenant";
import { mintAgentToken } from "@lib/agent/token";

export async function POST() {
  const t = await currentTenant();
  if (!t) return NextResponse.json({ error: "unauthenticated" }, { status: 401 });
  return NextResponse.json({ token: await mintAgentToken(t.tenantId, t.userId), expiresIn: 900 });
}
```

Checkpoint: log in, then in the browser console:

```
await (await fetch("/api/agent-token", { method: "POST" })).json()
```

Expected: `{ token: "eyJ...", expiresIn: 900 }`. Paste the token at `jwt.io` — payload shows `tenant_id` and `sub`. The Python service must accept it: `curl localhost:8000/connections -H "Authorization: Bearer <token>" → []` (not 401).

14. Agent client, connections and chat

14.1 Generate the typed client

```
# agent must be running
pnpm openapi-typescript http://localhost:8000/openapi.json -o src/lib/agent/openapi.d.ts
```

Add to package.json scripts: "gen:agent": "openapi-typescript \$AGENT_URL/openapi.json -o src/lib/agent/openapi.d.ts". Run it in CI so a contract change breaks the build, not production.

File: src/lib/agent/client.ts

```
import createClient from "openapi-fetch";
import type { paths } from "./openapi";

export function agentClient(baseUrl: string, token: string) {
  return createClient<paths>({ baseUrl, headers: { Authorization: `Bearer ${token}` } });
}
```

14.2 Connections — server-side proxy route

The browser never posts a DSN to the Python service directly; it posts to Next.js, which forwards with a server-minted token. This keeps NEXT_PUBLIC_AGENT_URL usable for SSE only and lets you add audit logging in one place.

File: src/app/api/connections/route.ts

```
import { NextResponse } from "next/server";
import { z } from "zod";
import { currentTenant } from "@lib/tenant";
import { mintAgentToken } from "@lib/agent/token";
import { agentClient } from "@lib/agent/client";
import { serverEnv } from "@lib/env";

const Body = z.object({
  name: z.string().min(1).max(200),
  kind: z.enum(["postgres", "web"]),
  secret: z.record(z.string()),
});

async function client() {
  const t = await currentTenant();
  if (!t) return null;
  return agentClient(serverEnv().AGENT_URL, await mintAgentToken(t.tenantId, t.userId));
}

export async function GET() {
  const c = await client();
  if (!c) return NextResponse.json({ error: "unauthenticated" }, { status: 401 });
  const { data, error } = await c.GET("/connections");
  return error ? NextResponse.json(error, { status: 502 }) : NextResponse.json(data);
}

export async function POST(req: Request) {
  const c = await client();
  if (!c) return NextResponse.json({ error: "unauthenticated" }, { status: 401 });
  const parsed = Body.safeParse(await req.json());
  if (!parsed.success) return NextResponse.json({ error: parsed.error.flatten(), status: 400 });
  const { data, error, response } = await c.POST("/connections", { body: parsed.data });
  return error ? NextResponse.json(error, { status: response.status }) : NextResponse.json(data, { status: 201 });
}
```

File: src/hooks/useConnections.ts

```
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";

export type Connection = { id: string; name: string; kind: "postgres" | "web"; has_schema_cache: boolean };

export function useConnections() {
  return useQuery<Connection[]>({ queryKey: ["connections"],
    queryFn: async () => { const r = await fetch("/api/connections"); if (!r.ok) throw new Error(await r.text()); return r.json(); } });
}

export function useCreateConnection() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (body: { name: string; kind: "postgres" | "web"; secret: Record<string, string> }) => {
      const r = await fetch("/api/connections", { method: "POST", headers: { "content-type": "application/json" }, body: JSON.stringify(body) });
      if (!r.ok) throw new Error((await r.json()).error ?? "failed");
      return r.json() as Promise<Connection>;
    },
    onSuccess: () => qc.invalidateQueries({ queryKey: ["connections"] }),
  });
}
```

File: src/components/connections/ConnectionForm.tsx

```

"use client";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { useCreateConnection } from "@hooks/useConnections";

const Schema = z.object({ name: z.string().min(1), dsn: z.string().startsWith("postgres") });
type Values = z.infer<typeof Schema>;

export function ConnectionForm() {
  const { register, handleSubmit, formState: { errors, isSubmitting }, reset } = useForm<Values>({ resolver: zodResolver(Schema) });
  const create = useCreateConnection();
  return (
    <form onSubmit={handleSubmit(async (v) => { await create.mutateAsync({ name: v.name, kind: "postgres", secret: { dsn: v.dsn } }); reset();
  })}
      className="space-y-3 max-w-lg">
      <Input placeholder="Connection name" {...register("name")} />
      {errors.name && <p className="text-sm text-red-600">{errors.name.message}</p>}
      <Input placeholder="postgres+psycopg://analyst_ro:...@host:5432/db" {...register("dsn")} />
      {errors.dsn && <p className="text-sm text-red-600">{errors.dsn.message}</p>}
      {create.error && <p className="text-sm text-red-600">{String(create.error.message)}</p>}
      <Button type="submit" disabled={isSubmitting}>Test & save</Button>
    </form>
  );
}

```

Checkpoint: on `/connections`, submit the demo DSN from section 5.5. The list refreshes with `demo` and the Python `connections` table has one row with `tenant_id` equal to your Supabase tenant id.

14.3 SSE parser for POST streams

`EventSource` only supports GET. Parse the stream by hand.

File: `src/lib/agent/sse.ts`

```

export type AgentEvent =
  | { type: "status"; data: { stage: string } }
  | { type: "sql"; data: { sql: string } }
  | { type: "rows"; data: { columns: string[]; rows: unknown[][]; truncated: boolean } }
  | { type: "token"; data: { text: string } }
  | { type: "done"; data: { run_id: string; duration_ms: number } }
  | { type: "error"; data: { message: string } };

export async function* streamRun(baseUrl: string, token: string,
  body: { connection_id: string; thread_id: string; question: string }, signal?: AbortSignal): AsyncGenerator<AgentEvent> {
  const res = await fetch(`${baseUrl}/runs`, {
    method: "POST", signal,
    headers: { Authorization: `Bearer ${token}`, "content-type": "application/json", accept: "text/event-stream" },
    body: JSON.stringify(body),
  });
  if (!res.ok || !res.body) throw new Error(`agent ${res.status}: ${await res.text()}`);
  const reader = res.body.getReader();
  const dec = new TextDecoder();
  let buf = "";
  for (;;) {
    const { value, done } = await reader.read();
    if (done) break;
    buf += dec.decode(value, { stream: true });
    let idx: number;
    while ((idx = buf.indexOf("\n\n")) >= 0) {
      const block = buf.slice(0, idx); buf = buf.slice(idx + 2);
      let type = "message", data = "";
      for (const line of block.split("\n")) {
        if (line.startsWith("event:")) type = line.slice(6).trim();
        else if (line.startsWith("data:")) data += line.slice(5).trim();
      }
      if (data) yield { type, data: JSON.parse(data) } as AgentEvent;
    }
  }
}

```

14.4 Run state machine hook

File: `src/hooks/useRun.ts`

```

"use client";
import { useCallback, useRef, useState } from "react";
import { streamRun, type AgentEvent } from "@lib/agent/sse";
import { clientEnv } from "@lib/env";

export type RunState = {
  stage: "idle" | "router" | "sql_gen" | "sql_guard" | "db_exec" | "web_tool" | "answer" | "done" | "error";
  sql?: string; columns?: string[]; rows?: unknown[][]; truncated?: boolean;
  answer: string; error?: string; runId?: string; durationMs?: number;
};

const initial: RunState = { stage: "idle", answer: "" };

async function getToken(): Promise<string> {
  const r = await fetch("/api/agent-token", { method: "POST" });
  if (!r.ok) throw new Error("could not get agent token");
  return (await r.json()).token;
}

export function useRun() {
  const [state, setState] = useState<RunState>(initial);
  const abort = useRef<AbortController | null>(null);

  const ask = useCallback(async (connectionId: string, threadId: string, question: string) => {
    abort.current?.abort();
    abort.current = new AbortController();
    setState({ ...initial, stage: "router" });
    try {
      const token = await getToken();
      for await (const ev of streamRun(clientEnv.NEXT_PUBLIC_AGENT_URL, token,
        { connection_id: connectionId, thread_id: threadId, question }, abort.current.signal)) {
        setState((s) => reduce(s, ev));
      }
    } catch (e) {
      if ((e as Error).name !== "AbortError") setState((s) => ({ ...s, stage: "error", error: (e as Error).message }));
    }
  }, []);

  const cancel = useCallback(() => abort.current?.abort(), []);
  return { state, ask, cancel };
}

function reduce(s: RunState, ev: AgentEvent): RunState {
  switch (ev.type) {
    case "status": return { ...s, stage: ev.data.stage as RunState["stage"] };
    case "sql": return { ...s, sql: ev.data.sql };
    case "rows": return { ...s, ...ev.data };
    case "token": return { ...s, answer: s.answer + ev.data.text };
    case "done": return { ...s, stage: "done", runId: ev.data.run_id, durationMs: ev.data.duration_ms };
    case "error": return { ...s, stage: "error", error: ev.data.message };
  }
}

```

14.5 Chat UI

File: src/components/chat/ChatPanel.tsx

```
"use client";
import { useState } from "react";
import { useRun } from "@hooks/useRun";
import { useConnections } from "@hooks/useConnections";
import { Button } from "@components/ui/button";
import { Textarea } from "@components/ui/textarea";
import { ResultTable } from "../ResultTable";
import { SqlDisclosure } from "../SqlDisclosure";

const STAGE_LABEL: Record<string, string> = {
  router: "Understanding the question...", sql_gen: "Writing the query...", sql_guard: "Checking the query...",
  db_exec: "Running on your database...", web_tool: "Fetching live data...", answer: "Writing the answer...",
};

export function ChatPanel() {
  const { data: connections } = useConnections();
  const [connectionId, setConnectionId] = useState<string>("");
  const [question, setQuestion] = useState("");
  const [threadId] = useState(() => crypto.randomUUID());
  const { state, ask, cancel } = useRun();
  const busy = !["idle", "done", "error"].includes(state.stage);

  return (
    <div className="space-y-4 max-w-3xl">
      <select className="border rounded p-2" value={connectionId} onChange={(e) => setConnectionId(e.target.value)}>
        <option value="">Select a connection</option>
        {connections?.map((c) => <option key={c.id} value={c.id}>{c.name} ({c.kind})</option>)}
      </select>
      <Textarea value={question} onChange={(e) => setQuestion(e.target.value)} placeholder="Which dealer sold the most batteries last month?" />
      <div className="flex gap-2">
        <Button disabled={!connectionId || question.length < 3 || busy} onClick={() => ask(connectionId, threadId, question)}>Ask</Button>
        {busy && <Button variant="outline" onClick={cancel}>Cancel</Button>}
      </div>
      {busy && <p className="text-sm text-muted-foreground animate-pulse">{STAGE_LABEL[state.stage] ?? "Working..."}</p>}
      {state.error && <p className="text-sm text-red-600">{state.error}</p>}
      {state.answer && <p className="whitespace-pre-wrap">{state.answer}</p>}
      {state.columns && <ResultTable columns={state.columns} rows={state.rows ?? []} truncated={!state.truncated} />}
      {state.sql && <SqlDisclosure sql={state.sql} />}
      {state.stage === "done" && <p className="text-xs text-muted-foreground">run {state.runId} {state.durationMs} ms</p>}
    </div>
  );
}
```

File: src/components/chat/ResultTable.tsx

```
export function ResultTable({ columns, rows, truncated }: { columns: string[]; rows: unknown[][]; truncated: boolean }) {
  return (
    <div className="overflow-auto border rounded">
      <table className="text-sm w-full">
        <thead className="bg-muted"><tr>{columns.map((c) => <th key={c} className="text-left p-2">{c}</th>)}</tr></thead>
        <tbody>{rows.map((r, i) => <tr key={i} className="border-t">{r.map((v, j) => <td key={j} className="p-2">{String(v ?? "")}</td>)}</tr>)}
      </tbody>
    </table>
    {truncated && <p className="p-2 text-xs text-muted-foreground">Showing the first rows only.</p>}
  </div>
);
}
```

File: src/components/chat/SqlDisclosure.tsx

```
export function SqlDisclosure({ sql }: { sql: string }) {
  return (
    <details className="text-sm">
      <summary className="cursor-pointer text-muted-foreground">Show SQL</summary>
      <pre className="mt-2 p-3 bg-muted rounded overflow-auto text-xs">{sql}</pre>
    </details>
  );
}
```

Checkpoint (end-to-end): log in → /connections add demo DB → /chat ask "How many dealers do we have?" You should see the stage label change, then the answer, table and "Show SQL". The Python runs table gains a done row with your tenant id.

Common errors

Symptom	Fix
Nothing streams; everything appears at the end	Behind Caddy: flush_interval -1 (section 9.3). In dev, confirm res.body exists and you are not going through Next.js rewrites (which buffer)
CORS error in console	Add the Next.js origin to allow_origins in app/main.py
401 after ~15 minutes	Token expired mid-session; useRun fetches a fresh token per question, so this only affects a single very long run — raise expiry to 30m if runs exceed 15 minutes
crypto.randomUUID is not a function	Page served over plain HTTP on a non-localhost host; use HTTPS or a uuid library

15. Frontend testing, billing and deployment

15.1 Vitest

File: `vitest.config.ts`

```
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";
export default defineConfig({
  plugins: [react()],
  test: { environment: "jsdom", setupFiles: ["/tests/setup.ts"], include: ["tests/unit/**/*.test.{ts,tsx}"],
    coverage: { reporter: ["text", "html"], include: ["src/lib/**", "src/hooks/**"] } },
  resolve: { alias: { "@": "/src" } },
});
```

File: `tests/setup.ts` — `import "@testing-library/jest-dom/vitest";`

File: `tests/unit/sse.test.ts`

```
import { describe, it, expect, vi } from "vitest";
import { streamRun } from "@lib/agent/sse";

function body(text: string) {
  return new ReadableStream({ start(c) { c.enqueue(new TextEncoder().encode(text)); c.close(); } });
}

describe("streamRun", () => {
  it("parses event/data blocks, including split chunks", async () => {
    vi.stubGlobal("fetch", vi.fn().mockResolvedValue(new Response(
      body('event: status\ndata: {"stage":"router"}\n\nevent: token\ndata: {"text":"hi"}\n\n'), { status: 200 })));
    const out = [];
    for await (const ev of streamRun("http://x", "t", { connection_id: "c", thread_id: "th", question: "q" })) out.push(ev);
    expect(out).toEqual([
      { type: "status", data: { stage: "router" } },
      { type: "token", data: { text: "hi" } }
    ]);
  });
  it("throws on non-2xx", async () => {
    vi.stubGlobal("fetch", vi.fn().mockResolvedValue(new Response("nope", { status: 401 })));
    await expect(async () => { for await (const _ of streamRun("http://x", "t", { connection_id: "c", thread_id: "th", question: "q" })) {} })
      .rejects.toThrow(/401/);
  });
});
```

Run: `pnpm vitest run --coverage`. Target: 90% on `src/lib/agent/`, 80% on `src/hooks/`.

15.2 Playwright e2e

File: `tests/e2e/chat.spec.ts`

```
import { test, expect } from "@playwright/test";

test("ask a question end to end", async ({ page }) => {
  await page.goto("/login");
  await page.fill("input[type=email]", process.env.E2E_EMAIL!);
  await page.fill("input[type=password]", process.env.E2E_PASSWORD!);
  await page.click("button[type=submit]");
  await page.goto("/chat");
  await page.selectOption("select", { index: 1 });
  await page.fill("textarea", "How many dealers do we have?");
  await page.click("text=Ask");
  await expect(page.locator("text=Show SQL")).toBeVisible({ timeout: 60_000 });
  await expect(page.locator("table")).toContainText("3");
});
```

Run: `pnpm playwright test` with the agent, Compose DBs and `pnpm dev` running. Keep this to 3-5 golden journeys; the Python evals cover answer quality.

15.3 Billing hooks (phase 6)

- `/billing` page: Razorpay Checkout for a subscription plan; on success the webhook route sets `tenants.plan` in Supabase **and** calls a small admin endpoint on the Python service (`PATCH /admin/tenants/{id}` guarded by a service key) to set `daily_token_budget`. Add that endpoint when you reach this phase; do not expose it through the customer JWT.
- Verify the webhook signature with `RAZORPAY_WEBHOOK_SECRET` before touching any table.
- `/usage` page: a Next.js route handler queries the Python service for run aggregates (add `GET /usage/summary` there) and renders tokens, runs and rows for the last 30 days.

15.4 Dockerfile and deploy

File: `next.config.ts` — add `output: "standalone"`.

File: `Dockerfile`

```

FROM node:20-alpine AS deps
WORKDIR /app
RUN corepack enable
COPY package.json pnpm-lock.yaml ./
RUN pnpm install --frozen-lockfile

FROM node:20-alpine AS build
WORKDIR /app
RUN corepack enable
COPY --from=deps /app/node_modules ./node_modules
COPY . .
ARG NEXT_PUBLIC_SUPABASE_URL NEXT_PUBLIC_SUPABASE_ANON_KEY NEXT_PUBLIC_AGENT_URL
ENV NEXT_PUBLIC_SUPABASE_URL=$NEXT_PUBLIC_SUPABASE_URL NEXT_PUBLIC_SUPABASE_ANON_KEY=$NEXT_PUBLIC_SUPABASE_ANON_KEY
NEXT_PUBLIC_AGENT_URL=$NEXT_PUBLIC_AGENT_URL
RUN pnpm build

FROM node:20-alpine AS run
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/.next/standalone ./
COPY --from=build /app/.next/static ./next/static
COPY --from=build /app/public ./public
USER node
EXPOSE 3000
CMD ["node", "server.js"]

```

Add to the production `docker-compose.yml` from section 9.3:

```

web:
  image: ghcr.io/itarangit/analyst-web:${WEB_TAG:-latest}
  env_file: .env.web
  restart: unless-stopped

```

And to the `Caddyfile`:

```

app.yourdomain.com {
  reverse_proxy web:3000
}

```

CI mirrors section 9.5: `pnpm lint && pnpm vitest run && pnpm build`, then `docker/build-push-action` with the three `NEXT_PUBLIC_*` build args.

15.5 Frontend readiness checklist

- [] `SUPABASE_SERVICE_ROLE_KEY` and `AGENT_JWT_SECRET` appear only in server files (`grep -r` under `src/app/api` and `src/lib/agent/token.ts`)
- [] RLS enabled on `tenants` and `memberships`; a second test user cannot read the first user's tenant
- [] Agent token expiry ≤ 15 min; minted only from `/api/agent-token`
- [] `openapi.d.ts` regenerated in CI; type errors fail the build
- [] SSE streams progressively through Caddy in production (watch the stage label change)
- [] `ResultTable` renders 500 rows without freezing (virtualise if not)
- [] Cancel button aborts the fetch and the Python `runs` row ends as `error`, not `running` forever
- [] Login, connection create, one question — all three pass in Playwright against the staging stack

16. Appendix

16.1 Command cheat sheet

```
# local infra
docker compose -f docker-compose.dev.yml up -d
# run
uvicorn app.main:app --reload --port 8000
# migrations
alembic revision --autogenerate -m "msg" && alembic upgrade head
# tests
pytest -m "not integration" ; pytest -m integration
# evals
TOKEN=... CONN=... python evals/run_evals.py
# lint
ruff check app tests && ruff format app tests
# lock deps
pip-compile --extra dev -o requirements.lock pyproject.toml
# prod
docker compose pull agent && docker compose up -d agent
```

16.2 Extension points (phase 5)

- **File tool:** add `kind="file"` to `Connection`; a `DuckDBConnector` implementing `describe_schema` and `run_select` over `read_csv_auto()`; the guard's dialect becomes `"duckdb"`. Nothing else in the graph changes.
- **Charts:** in `answer_node`, when `rows` is numeric-heavy, emit a second structured output `{chart: {type, x, y}}` and stream it as a `chart` event.
- **Queue:** move `stream_run`'s try-block into an `arq` task; the route enqueues and reads `run:<id>` from a Redis stream.
- **More connectors:** MySQL and MSSQL are the same `PostgresConnector` with a different DSN and `read=` dialect for sqlglot. Add one only when a paying customer asks.

16.3 Troubleshooting index

Area	Symptom	Section
Setup	psycopg build fails	2.4
Settings	ValidationError at startup	4.1
Migrations	empty autogenerate	5.3
Customer DB	DELETE succeeds on read-only role	5.5
Auth	401 with a valid-looking token	check <code>JWT_SECRET</code> matches Next.js; <code>jwt.decode</code> error is logged
Vault	decrypt failed	7.10
Runs	no SSE events in browser	Caddy <code>flush_interval -1</code> , section 9.3
Evals	pass locally, fail in CI	CI uses the demo DB service; confirm seed ran